signal-processing: the Signal Processing program — user guide

signal-processing-ai999x 1.0.3, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Signal Processing covers sampling, Fourier series and transforms, the DFT and FFT, filters, convolution, z-transforms, modulation, communication links, RF transmission lines and radar. This guide is about `signal-processing`, the model as a self-contained program for Windows and Linux: 74 capabilities in 18 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/signal-processing-ai999x/); the Python package (`pip install signal-processing-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.4.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS; the newest version is always under https://downloads.ai999x.com/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Signal Processing	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-signal-processing-1.4.0-windows-x86_64.zip
Desktop app: Signal Processing	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-signal-processing-1.4.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-windows-x86_64.zip
Command line	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz

Desktop app

Signal Processing in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-signal-processing-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-signal-processing.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-signal-processing-1.4.0-linux-
x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-signal-processing-1.4.0-linux-x86_64.tar.gz
./ai999x-signal-processing-1.4.0-linux-x86_64/ai999x-signal-processing
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.4.0-linux-x86_64.tar.gz
./ai999x-1.4.0-linux-x86_64/ai999x
```

Command line — Linux

The bundle holds one program per model; `signal-processing` is this one. The checksum line should print OK.

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.4.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.4.0-linux-x86_64:$PATH"' >> ~/.bashrc && source
~/.bashrc
signal-processing --version
```

Command line — Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.4.0-windows-x86_64.zip"
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/$F" -OutFile $F
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F
SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.4.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User")
+ ";$D", "User")
signal-processing --version
```

In your browser

Open https://www.ai999x.com/models/signal-processing-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install signal-processing-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ signal-processing --version
signal-processing-ai999x 1.0.3 (Rust build, ax-engine 0.1.0)
```

2.2 Ask in words

Write the question as you would say it: the program reads the quantities and their units, picks the capability and answers.

```
$ signal-processing ask "What is the Nyquist rate for a signal with a maximum frequency of 4 kHz?"
Read as: Nyquist rate
fs_min = 8000 Hz
Ts_max = 0.000125 s
  by: Nyquist rate  (sampling theorem: fs >= 2 fmax)
  fmax = 4000 Hz
  assumes: fmax > 0; a baseband (lowpass) signal
```

2.3 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ signal-processing search "reflection coefficient, vswr and return loss of a load"
reflection_coefficient              Reflection coefficient, VSWR and return loss of a load
(transmission lines)
reflection_from_vswr                Reflection coefficient and return loss from the VSWR
(transmission lines)
vswr_from_reflection                VSWR and return loss from the reflection coefficient
(transmission lines)
input_impedance                     Input impedance of a terminated lossless line  (transmission
lines)
free_space_path_loss                Free-space path loss  (noise and link budgets)
window_coefficients                 Coefficients of a window function  (windows and spectral
analysis)
quarter_wave_transformer            Quarter-wave impedance transformer  (transmission lines)
bilinear_rc_lowpass                 Bilinear transform of a first-order low-pass filter  (digital
filters)
… 12 more
```

2.4 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ signal-processing list
alias_frequency                          Alias (apparent) frequency after sampling
am_modulation_index                      AM modulation index from the envelope
am_power                                 Power in an AM (DSB full-carrier) signal
amplitude_ratio_to_db                    Amplitude (voltage) ratio to decibels
analytic_envelope                        Analytic signal, Hilbert transform and envelope
bilinear_rc_lowpass                      Bilinear transform of a first-order low-pass filter
bin_frequency                            Frequency of a DFT bin
bpsk_ber                                 Bit error rate of BPSK (and coherent BFSK) in AWGN
… 66 more
```

2.5 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ signal-processing info reflection_coefficient
Reflection coefficient, VSWR and return loss of a load
  Gamma = (ZL - Z0)/(ZL + Z0); VSWR = (1 + |Gamma|)/(1 - |Gamma|)
  in   RL       load resistance [ohm]
  in   Z0       characteristic impedance of the line [ohm]
  in   XL       load reactance (0 if not given) [ohm]
  out  gamma_mag |Gamma| []
  out  gamma_angle_deg angle of Gamma in degrees []
  out  gamma_re real part of Gamma []
  out  gamma_im imaginary part of Gamma []
  out  vswr     voltage standing wave ratio []
  out  return_loss_db return loss in dB []
  out  mismatch_loss_db mismatch loss in dB []
```

2.6 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ signal-processing solve reflection_coefficient "RL=75 ohm" "XL=25 ohm" "Z0=50 ohm"
gamma_mag = 0.2773500981
gamma_angle_deg = 33.69006753
gamma_re = 0.2307692308
gamma_im = 0.1538461538
vswr = 1.767591879
return_loss_db = 11.13943352
mismatch_loss_db = 0.3476210626
  by: Reflection coefficient, VSWR and return loss of a load  (Gamma = (ZL - Z0)/(ZL + Z0); VSWR =
(1 + |Gamma|)/(1 - |Gamma|))
  RL = 75 ohm
  XL = 25 ohm
  Z0 = 50 ohm
  assumes: RL > 0; Z0 real (lossless line)
```

2.7 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ signal-processing solve reflection_coefficient "RL=75 ohm" "XL=25 ohm" "Z0=50 ohm" --json
{"outcome":"SOLVED","message":"","value":0.2773500981126146,"unit":"","solved_for":"gamma_mag","si":
[["RL",75.0],["XL",25.0],["Z0",50.0]],"values":[["gamma_mag",0.2773500981126146,""],
["gamma_angle_deg",33.690067525979785,""],["gamma_re",0.23076923076923078,""],
["gamma_im",0.15384615384615385,""],["vswr",1.7675918792439984,""],
["return_loss_db",11.139433523068368,""],
["mismatch_loss_db",0.34762106259211917,""]],"target":"gamma_mag","phrase":"|Gamma|","assumptions":
["RL > 0","Z0 real (lossless line)"],"name":"Reflection coefficient, VSWR and return loss of a
load","equation":"Gamma = (ZL - Z0)/(ZL + Z0); VSWR = (1 + |Gamma|)/(1 - |Gamma|)"}
```

2.8 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ signal-processing solve reflection_coefficient "RL=75 ohm" "XL=25 ohm"
NEEDS_INPUT: reflection_coefficient needs Z0 (characteristic impedance of the line). Example:
solve('reflection_coefficient', RL='75 ohm', XL='25 ohm', Z0='50 ohm').
```

2.9 Draw the figures

The live figures for your own inputs, written as SVG files into the folder given with --out (here the current folder); open them in any browser. Without --out the SVG text comes back as JSON.

```
$ signal-processing figures nyquist_rate "fmax=4 kHz" --out .
[
  {
    "slug": "nyquist-rate-live-1",
    "title": "Nyquist rate (minimum sampling rate) as highest frequency in the signal varies",
    "caption": "Nyquist rate (minimum sampling rate) as fmax runs from 1 to 7 kHz, every other
input as you entered it (41 solutions). Marked: your fmax = 4 kHz gives fs_min = 8000. Your inputs:
fmax = 4 kHz.",
    "file": ".\\nyquist-rate-live-1.svg"
  }
]
```

2.10 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | signal-processing batch

`cases.jsonl`:

```
{"key": "reflection_coefficient", "inputs": {"RL": "75 ohm", "XL": "25 ohm", "Z0": "50 ohm"}}
{"key": "nyquist_rate", "inputs": {"fmax": "4 kHz"}}
```

```
$ signal-processing batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":0.2773500981126146,"unit":"","solved_for":"gamma_mag","si":
[["RL",75.0],["XL",25.0],["Z0",50.0]],"values":[["gamma_mag",0.2773500981126146,""],
["gamma_angle_deg",33.690067525979785,""],["gamma_re",0.23076923076923078,""],
["gamma_im",0.15384615384615385,""],["vswr",1.7675918792439984,""],
["return_loss_db",11.139433523068368,""],
["mismatch_loss_db",0.34762106259211917,""]],"target":"gamma_mag","phrase":"|Gamma|","assumptions":
["RL > 0","Z0 real (lossless line)"],"name":"Reflection coefficient, VSWR and return loss of a
load","equation":"Gamma = (ZL - Z0)/(ZL + Z0); VSWR = (1 + |Gamma|)/(1 - |Gamma|)"}
{"outcome":"SOLVED","message":"","value":8000.0,"unit":"Hz","solved_for":"fs_min","si":
[["fmax",4000.0]],"values":[["fs_min",8000.0,"Hz"],
["Ts_max",0.000125,"s"]],"target":"fs_min","phrase":"Nyquist rate (minimum sampling
rate)","assumptions":["fmax > 0","a baseband (lowpass) signal"],"name":"Nyquist
rate","equation":"sampling theorem: fs >= 2 fmax"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `signal-processing info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]" or "v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`signal-processing solve ... --json` prints one answer as one line of JSON. `signal-processing batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | signal-processing batch`. `signal-processing catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["signal-processing", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`signal-processing figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

```
$ signal-processing figures nyquist_rate "fmax=4 kHz" --out .
[
  {
    "slug": "nyquist-rate-live-1",
    "title": "Nyquist rate (minimum sampling rate) as highest frequency in the signal varies",
    "caption": "Nyquist rate (minimum sampling rate) as fmax runs from 1 to 7 kHz, every other
input as you entered it (41 solutions). Marked: your fmax = 4 kHz gives fs_min = 8000. Your inputs:
fmax = 4 kHz.",
    "file": ".\\nyquist-rate-live-1.svg"
  }
]
```

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `signal_processing_ai999x.wasm` (https://www.ai999x.com/compute/rust/signal_processing_ai999x.wasm), with a small loader, `ax-wasm.js`

(https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("signal_processing_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `signal-processing info <id>` shows the same inputs and outputs in the terminal.

Analog filters (3)

Magnitude of an analog Butterworth low-pass response — `butterworth_magnitude`

|H(f)| = 1/sqrt(1 + (f/fc)^(2N)) of an N-th order Butterworth low-pass filter, and the attenuation in dB.

$$|H(f)| = \frac{1}{\sqrt{1 + (f/f_c)^{2N}}}$$

symbol		meaning	SI unit
`N`	in	filter order	—
`fc`	in	cutoff (-3 dB) frequency	Hz
`f`	in	frequency	Hz
`magnitude`	out	|H(f)|	—
`attenuation_db`	out	attenuation in dB (positive)	—

Applies when: N >= 1; fc > 0; f >= 0.

```
$ signal-processing solve butterworth_magnitude N=4 "fc=1 kHz" "f=2 kHz"
magnitude = 0.06237828616
attenuation_db = 24.09933123
  by: Magnitude of an analog Butterworth low-pass response  (the Butterworth magnitude-squared
function)
  fc = 1000 Hz
  f = 2000 Hz
  assumes: N >= 1; fc > 0; f >= 0
```

What the worked example means: This calculation determines the magnitude response of an analog Butterworth low-pass filter. Using the filter order 4, the cutoff frequency 1 kHz, and the specific input frequency 2 kHz, the model computes the normalized gain 0.0623783. This value represents the ratio of output amplitude to input amplitude at that frequency. The corresponding attenuation in decibels, 24.0993,

indicates how much signal strength is lost relative to the passband. A lower magnitude signifies greater attenuation, which is typical for frequencies above the cutoff point.

Minimum order of a analog Butterworth filter — `butterworth_order_analog`

The smallest order meeting a passband edge with at most Rp dB attenuation and a stopband edge with at least As dB.

$$N \geq \frac{\log_{10} \frac{10^{A_s/10}-1}{10^{R_p/10}-1}}{2\log_{10}(\omega_s/\omega_p)}$$

symbol		meaning	SI unit
`fp`	in	passband edge frequency	Hz
`fstop`	in	stopband edge frequency	Hz
`Rp`	in	maximum passband attenuation (ripple) in dB	—
`As`	in	minimum stopband attenuation in dB	—
`order`	out	minimum filter order	—
`type`	out	lowpass (fp < fstop) or highpass (fp > fstop)	—

Applies when: 0 < Rp < As; fp != fstop.

```
$ signal-processing solve butterworth_order_analog "fp=1 kHz" "fstop=2 kHz" Rp=1 As=40
order = 8
type = lowpass
  by: Minimum order of a analog Butterworth filter  (N = ceil( log10((10^(As/10)-1)/(10^(Rp/10)-1))
/ (2 log10(ws/wp)) ))
  fp = 1000 Hz
  fstop = 2000 Hz
  Rp = 1
  As = 40
  assumes: 0 < Rp < As; fp != fstop
```

What the worked example means: This calculation determines the smallest integer order N required for an analog Butterworth filter to meet specific passband and stopband attenuation constraints. Given a maximum passband ripple of 1 dB at the passband edge frequency 1 kHz Hz and a minimum stopband attenuation of 40 dB at the stopband edge frequency 2 kHz Hz, the resulting minimum filter order is 8. This ensures the filter provides a maximally flat magnitude response in the passband while achieving the necessary signal rejection in the stopband.

Minimum order of a analog Chebyshev type I filter — `chebyshev1_order_analog`

The smallest order meeting a passband edge with at most Rp dB attenuation and a stopband edge with at least As dB.

$$N \geq \frac{\cosh^{-1}\sqrt{\frac{10^{A_s/10}-1}{10^{R_p/10}-1}}}{\cosh^{-1}(\omega_s/\omega_p)}$$

symbol		meaning	SI unit
`fp`	in	passband edge frequency	Hz
`fstop`	in	stopband edge frequency	Hz
`Rp`	in	maximum passband attenuation (ripple) in dB	—
`As`	in	minimum stopband attenuation in dB	—
`order`	out	minimum filter order	—
`type`	out	lowpass (fp < fstop) or highpass (fp > fstop)	—

Applies when: 0 < Rp < As; fp != fstop.

```
$ signal-processing solve chebyshev1_order_analog "fp=1 kHz" "fstop=2 kHz" Rp=1 As=40
order = 5
type = lowpass
  by: Minimum order of a analog Chebyshev type I filter  (N = ceil(
acosh(sqrt((10^(As/10)-1)/(10^(Rp/10)-1))) / acosh(ws/wp) ))
  fp = 1000 Hz
  fstop = 2000 Hz
  Rp = 1
  As = 40
  assumes: 0 < Rp < As; fp != fstop
```

What the worked example means: This calculation determines the smallest integer order required for an analog Chebyshev Type I filter to meet specific attenuation constraints. By analyzing the passband edge frequency of 1 kHz, the stopband edge frequency of 2 kHz, the maximum passband ripple of 1, and the minimum stopband attenuation of 40, the algorithm computes that a filter of order 5 is necessary. This result indicates the complexity needed to achieve the desired transition band sharpness while maintaining the specified ripple characteristics, identifying the filter type as lowpass.

Antennas (1)

Gain of a parabolic dish (aperture) antenna — `dish_antenna_gain`

The gain eta (pi D / lambda)^2 of a circular aperture antenna of diameter D and aperture efficiency eta, in dBi, and its effective aperture.

$$G = \eta \left(\frac{\pi D}{\lambda} \right)^2$$

symbol		meaning	SI unit
`D`	in	dish diameter	m
`f`	in	frequency	Hz
`eta`	in	aperture efficiency (0 < eta <= 1)	—
`G_dbi`	out	gain in dBi	—
`G`	out	gain as a ratio	—
`Ae`	out	effective aperture	m^2

Applies when: 0 < eta <= 1; D >= one wavelength.

```
$ signal-processing solve dish_antenna_gain "D=3 m" "f=10 GHz" eta=0.55
G_dbi = 47.35263538
G = 54358.00866
Ae = 3.887720909 m^2
  by: Gain of a parabolic dish (aperture) antenna  (G = eta (pi D / lambda)^2 = 4 pi Ae / lambda^2)
  D = 3 m
  f = 1e+10 Hz
  eta = 0.55
  assumes: 0 < eta <= 1; D >= one wavelength
```

What the worked example means: This calculation determines the directional gain of a parabolic dish antenna using its diameter and operating frequency. Given a dish diameter of 3 m and a frequency of 10 GHz, combined with an aperture efficiency of 0.55, the resulting gain expressed as a ratio is 54358. This translates to 47.3526 in decibels relative to an isotropic source. The effective aperture, which represents the area that captures power from the incoming wave, is calculated as 3.88772 m^2. Higher gain values indicate a narrower beamwidth and greater ability to focus energy in a specific direction.

Communication links (2)

Minimum SNR for a data rate (Shannon limit) — `shannon_min_snr`

The smallest SNR = 2^(C/B) - 1 that lets an AWGN channel of bandwidth B carry C bit/s, and the corresponding minimum Eb/N0.

$$\mathrm{SNR}_{\min} = 2^{C/B} - 1, \quad \left(\frac{E_b}{N_0}\right)_{\min} = \frac{2^{C/B} - 1}{C/B}$$

symbol		meaning	SI unit
`C`	in	data rate in bit/s	—
`B`	in	bandwidth	Hz
`snr_db`	out	minimum SNR in dB	—
`snr`	out	minimum SNR (linear)	—
`ebn0_min_db`	out	minimum Eb/N0 in dB	—

Applies when: additive white Gaussian noise; C/B <= 200.

```
$ signal-processing solve shannon_min_snr "B=2 MHz" C=10000000
snr_db = 14.91361694
snr = 31
ebn0_min_db = 7.923916895
  by: Minimum SNR for a data rate (Shannon limit)  (invert C = B log2(1 + SNR))
  B = 2000000 Hz
  C = 10000000
  assumes: additive white Gaussian noise; C/B <= 200
```

What the worked example means: This calculation determines the theoretical limit for reliable data transmission over a noisy channel. Given a data rate of 1e+07 and a bandwidth of 2 MHz, it computes the smallest linear signal-to-noise ratio of 31 needed to achieve that throughput without error. The result also

provides the equivalent minimum energy per bit relative to noise density as 7.92392 decibels, which helps engineers assess if their link budget meets these fundamental physical constraints.

Shannon capacity of an AWGN channel — `shannon_capacity`

The channel capacity C = B log2(1 + SNR) of a band-limited additive white Gaussian noise channel and the spectral efficiency C/B.

$$C = B \log_2(1 + \mathrm{SNR})$$

symbol		meaning	SI unit
`B`	in	bandwidth	Hz
`snr_db`	in	signal-to-noise ratio in dB	—
`C`	out	capacity in bit/s	—
`efficiency`	out	spectral efficiency in bit/s per Hz	—

Applies when: additive white Gaussian noise; SNR stated in dB.

```
$ signal-processing solve shannon_capacity "B=3 kHz" snr_db=30
C = 29901.67878
efficiency = 9.967226259
  by: Shannon capacity of an AWGN channel  (Shannon-Hartley theorem)
  B = 3000 Hz
  snr_db = 30
  assumes: additive white Gaussian noise; SNR stated in dB
```

What the worked example means: This calculation determines the maximum theoretical data rate for a band-limited additive white Gaussian noise channel using the Shannon-Hartley theorem. Given a bandwidth of 3 kHz and a signal-to-noise ratio of 30, the resulting channel capacity is 29901.7. The corresponding spectral efficiency, which represents the bits per second transmitted per Hertz of bandwidth, is 9.96723. This value sets the fundamental upper limit on reliable communication speed for these specific channel conditions.

Convolution and correlation (3)

Circular (cyclic) convolution — `circular_convolution`

The N-point circular convolution y[n] = sum_k x[k] h[(n-k) mod N] of two sequences of length N.

$$y[n] = \sum_{k=0}^{N-1} x[k]\, h[(n-k) \bmod N]$$

symbol		meaning	SI unit
`x`	in	first sequence (length N)	—
`h`	in	second sequence (length N)	—
`y`	out	circular convolution y[0..N-1]	—

Applies when: both sequences have length N.

```
$ signal-processing solve circular_convolution "x=[1, 2, 3, 4]" "h=[1, 0, 0, 1]"
y = [3.0,5.0,7.0,5.0]
  by: Circular (cyclic) convolution  (direct cyclic sum (indices modulo N))
  assumes: both sequences have length N
```

What the worked example means: This calculation determines the N-point circular convolution of two sequences. Summing the product of the first sequence [1, 2, 3, 4] and the second sequence [1, 0, 0, 1] with indices wrapped modulo N gives the output sequence [3, 5, 7, 5]. This operation effectively models the interaction of periodic signals or systems where boundary conditions require index wrapping rather than zero padding.

Cross-correlation and the lag of its peak — `cross_correlation`

r_xy[l] = sum_n x[n+l] y[n] for every lag l, and the lag at which it is largest (the delay of x relative to y).

$$r_{xy}[\ell] = \sum_{n} x[n+\ell]\, y[n]$$

symbol		meaning	SI unit
`x`	in	first sequence x[n]	—
`y`	in	second (reference) sequence y[n]	—
`peak_lag`	out	lag of the largest correlation (positive: x lags y)	—
`peak_value`	out	largest correlation value	—
`r`	out	correlation at every lag	—
`lags`	out	the lags, -(Ny-1) .. Nx-1	—

Applies when: finite sequences starting at n = 0.

```
$ signal-processing solve cross_correlation "x=[0, 0, 1, 2, 1]" "y=[1, 2, 1]"
peak_lag = 2
peak_value = 6
r = [0.0,0.0,1.0,4.0,6.0,4.0,1.0]
lags = [-2,-1,0,1,2,3,4]
  by: Cross-correlation and the lag of its peak  (direct correlation sum over every lag (first lag
on ties))
  assumes: finite sequences starting at n = 0
```

What the worked example means: This calculation determines the similarity between two sequences by computing the sum of products for every possible displacement. The result identifies the specific shift, or lag, where this alignment is strongest. For these inputs, the highest correlation value is 6, which appears at the lag index 2. This lag represents the time delay of the first sequence relative to the second reference sequence.

Linear convolution of two sequences — `linear_convolution`

y[n] = sum_k x[k] h[n-k], the full linear convolution (length Nx + Nh - 1).

$$y[n] = \sum_{k} x[k]\, h[n-k]$$

symbol		meaning	SI unit
`x`	in	first sequence x[n]	—
`h`	in	second sequence h[n]	—
`y`	out	convolution y[n], n = 0 .. Nx+Nh-2	—
`length`	out	number of output samples	—

Applies when: finite sequences starting at n = 0.

```
$ signal-processing solve linear_convolution "x=[1, 2, 3]" "h=[1, -1]"
y = [1.0,1.0,1.0,-3.0]
length = 4
  by: Linear convolution of two sequences  (direct convolution sum (compensated))
  assumes: finite sequences starting at n = 0
```

What the worked example means: This calculation determines the complete response of a linear time-invariant system by convolving the input signal [1, 2, 3] with the impulse response [1, -1]. The resulting sequence [1, 1, 1, -3] represents how the filter modifies the input over time, extending beyond the duration of either original sequence. The total number of samples in this output is recorded as 4, which reflects the combined temporal support of both inputs.

DFT and FFT (5)

DFT spectrum of a sequence (FFT) — `dft_spectrum`

The whole N-point DFT of a real sequence: magnitude, phase, real and imaginary parts of every bin, and the strongest bin up to N/2.

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N},\ k = 0, \ldots, N-1$$

symbol		meaning	SI unit
`x`	in	samples x[0..N-1]	—
`peak_magnitude`	out	largest |X[k]| for 0 <= k <= N/2	—
`peak_bin`	out	bin of the largest magnitude	—
`magnitude`	out	|X[k]| for every bin	—
`phase`	out	phase of X[k] in radians for every bin	—
`real`	out	real parts	—
`imag`	out	imaginary parts	—

Applies when: real samples.

```
$ signal-processing solve dft_spectrum "x=[1, 2, 3, 4]"
peak_magnitude = 10
peak_bin = 0
magnitude = [10.0,2.8284271247461903,2.0,2.8284271247461903]
phase = [0.0,2.356194490192345,3.141592653589793,-2.356194490192345]
real = [10.0,-2.0,-2.0,-2.0]
imag = [0.0,2.0,0.0,-2.0]
  by: DFT spectrum of a sequence (FFT)  (radix-2 decimation-in-time FFT (N a power of two) or the
direct DFT sum)
  assumes: real samples
```

What the worked example means: This calculation transforms the time-domain samples [1, 2, 3, 4] into the frequency domain to reveal their spectral content. The result identifies the dominant frequency as bin 0, which has a magnitude of 10. Additionally, it provides the complete set of magnitude, phase, real, and imaginary values for every frequency bin, allowing for a detailed analysis of the signal's composition across the spectrum.

Frequency of a DFT bin — `bin_frequency`

The frequency k*fs/N that bin k of an N-point DFT represents (0 <= k <= N/2).

$$f_k = \frac{k\,f_s}{N}$$

symbol		meaning	SI unit
`k`	in	bin index	—
`fs`	in	sampling rate	Hz
`N`	in	number of DFT points	—
`f_k`	out	frequency of bin k	Hz

Applies when: 0 <= k <= N/2; fs > 0.

```
$ signal-processing solve bin_frequency "fs=48 kHz" N=1024 k=100
f_k = 4687.5 Hz
  by: Frequency of a DFT bin  (f_k = k fs / N)
  fs = 48000 Hz
  assumes: 0 <= k <= N/2; fs > 0
```

What the worked example means: This calculation determines the physical frequency represented by a specific index in the Discrete Fourier Transform. By using the sampling rate 48 kHz and the total number of points 1024, it maps the abstract bin index 100 to a concrete value of 4687.5 Hz. This result allows practitioners to interpret spectral peaks in terms of real-world cycles per second rather than just numerical indices.

Frequency resolution of an N-point DFT — `frequency_resolution`

The bin spacing fs/N of an N-point DFT or FFT and the record length N/fs it covers.

$$\Delta f = \frac{f_s}{N}$$

symbol		meaning	SI unit
`fs`	in	sampling rate	Hz
`N`	in	number of DFT points	—
`df`	out	frequency resolution (bin spacing)	Hz
`duration`	out	record length N/fs	s

Applies when: fs > 0; N >= 2.

```
$ signal-processing solve frequency_resolution "fs=48 kHz" N=1024
df = 46.875 Hz
duration = 0.02133333333 s
  by: Frequency resolution of an N-point DFT  (delta f = fs / N)
  fs = 48000 Hz
  assumes: fs > 0; N >= 2
```

What the worked example means: This calculation determines the spacing between adjacent frequency bins, which is 46.875 Hz, based on a sampling rate of 48 kHz and 1024 points. It also computes the total time duration covered by these samples, resulting in a record length of 0.0213333 s. These values define how finely frequencies can be distinguished and the window of time analyzed.

Inverse discrete Fourier transform — `inverse_dft`

x[n] = (1/N) sum_k X[k] exp(+j 2 pi k n / N), from the real and imaginary parts of X[k].

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k]\, e^{j2\pi kn/N}$$

symbol		meaning	SI unit
`re_`	in	real parts of X[0..N-1]	—
`im`	in	imaginary parts of X[0..N-1] (zeros if not given) (optional)	—
`real`	out	real parts of x[n]	—
`imag`	out	imaginary parts of x[n]	—

Applies when: both lists have the same length.

```
$ signal-processing solve inverse_dft "re_=[10, -2, -2, -2]" "im=[0, 2, 0, -2]"
real = [1.0,2.0,3.0,4.0]
imag = [0.0,0.0,0.0,0.0]
  by: Inverse discrete Fourier transform  (inverse FFT (conjugate twiddles) divided by N)
  assumes: both lists have the same length
```

What the worked example means: This calculation applies the inverse discrete Fourier transform to reconstruct the original time-domain signal from its frequency-domain representation. It takes the real parts [10, -2, -2, -2] and imaginary parts [0, 2, 0, -2] of the spectrum X[k] and computes the corresponding samples

of x[n]. The result consists of the real component [1, 2, 3, 4] and the imaginary component [0, 0, 0, 0], which together describe the amplitude and phase characteristics of the signal in the time domain.

One bin of the discrete Fourier transform — `dft_bin`

X[k] = sum_n x[n] exp(-j 2 pi k n / N) for one bin k: real and imaginary parts, magnitude and phase.

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}$$

symbol		meaning	SI unit
`x`	in	samples x[0..N-1]	—
`k`	in	bin index (0 <= k < N)	—
`magnitude`	out	|X[k]|	—
`phase`	out	phase of X[k] in radians (-pi, pi]	rad
`re`	out	real part of X[k]	—
`im`	out	imaginary part of X[k]	—

Applies when: 0 <= k < N.

```
$ signal-processing solve dft_bin "x=[1, 2, 3, 4]" k=1
magnitude = 2.828427125
phase = 2.35619449 rad
re = -2
im = 2
  by: One bin of the discrete Fourier transform  (direct evaluation of the DFT sum (compensated
sums, exact twiddles at quarter turns))
  assumes: 0 <= k < N
```

What the worked example means: This calculation determines the complex value for a single frequency component, specifically bin 1, within the discrete Fourier transform of the input sequence. By summing the products of each sample in [1, 2, 3, 4] with its corresponding exponential term, the algorithm isolates the contribution at this specific index. The result provides both the real part -2 and imaginary part 2, which together define the signal's strength as 2.82843 and its angular position as 2.35619 rad. This narrowband analysis allows for precise examination of one frequency without computing the entire spectrum.

Decibels and levels (8)

Amplitude (voltage) ratio to decibels — `amplitude_ratio_to_db`

A voltage, current or field amplitude ratio in decibels, 20*log10(V2/V1).

$$L = 20 \log_{10} \frac{V_2}{V_1} \text{ dB}$$

symbol		meaning	SI unit
`ratio`	in	amplitude ratio V2/V1 (a positive number)	—

symbol		meaning	SI unit
`dB`	out	level in decibels (dB)	—

Applies when: ratio > 0; both amplitudes across equal impedances.

```
$ signal-processing solve amplitude_ratio_to_db ratio=50
dB = 33.97940009
  by: Amplitude (voltage) ratio to decibels  (20 log10(ratio))
  ratio = 50
  assumes: ratio > 0; both amplitudes across equal impedances
```

What the worked example means: This calculation determines the decibel level for a given voltage or current amplitude ratio. Applying the formula twenty times the base ten logarithm of the input converts the linear ratio 50 into the logarithmic value 33.9794. This representation is useful because it compresses large ranges of signal strength into manageable numbers and aligns with human perception of loudness or intensity.

Decibels to amplitude (voltage) ratio — `db_to_amplitude_ratio`

The amplitude ratio V2/V1 = 10^(L/20) that a level of L dB represents.

$$\frac{V_2}{V_1} = 10^{L/20}$$

symbol		meaning	SI unit
`dB`	in	level in decibels	—
`ratio`	out	amplitude ratio V2/V1	—

Applies when: finite level.

```
$ signal-processing solve db_to_amplitude_ratio dB=6
ratio = 1.995262315
  by: Decibels to amplitude (voltage) ratio  (10^(L/20))
  dB = 6
  assumes: finite level
```

What the worked example means: This calculation determines the linear amplitude ratio V2 over V1 from a given decibel level. Applying the formula ten raised to the power of the level divided by twenty shows that a level of 6 results in an amplitude ratio of 1.99526. This value indicates how much larger or smaller the voltage signal is compared to the reference.

Decibels to power ratio — `db_to_power_ratio`

The power ratio P2/P1 = 10^(L/10) that a level of L dB represents.

$$\frac{P_2}{P_1} = 10^{L/10}$$

symbol		meaning	SI unit
`dB`	in	level in decibels	—
`ratio`	out	power ratio P2/P1	—

Applies when: finite level.

```
$ signal-processing solve db_to_power_ratio dB=23
ratio = 199.5262315
  by: Decibels to power ratio  (10^(L/10))
  dB = 23
  assumes: finite level
```

What the worked example means: This calculation determines the linear power ratio P2 over P1 from a given decibel level. Applying the method ten raised to the power of the input divided by ten converts the logarithmic scale back to a linear multiplier. The result shows that an input of 23 represents a power ratio of 199.526, indicating how many times greater or smaller the output power is compared to the reference.

Power in watts to dBm and dBW — `watts_to_dbm`

A power expressed as a level relative to 1 mW (dBm) and to 1 W (dBW).

$$P_{\mathrm{dBm}} = 10 \log_{10} \frac{P}{1\,\mathrm{mW}}$$

symbol		meaning	SI unit
`P`	in	power	W
`dBm`	out	power level in dBm	—
`dBW`	out	power level in dBW	—

Applies when: P > 0.

```
$ signal-processing solve watts_to_dbm "P=2 W"
dBm = 33.01029996
dBW = 3.010299957
  by: Power in watts to dBm and dBW  (10 log10(P / 1 mW))
  P = 2 W
  assumes: P > 0
```

What the worked example means: This calculation converts the input power value, 2 W, into logarithmic decibel units. The result is expressed as 33.0103, which indicates the signal strength relative to a reference of one milliwatt, and as 3.0103, which uses one watt as the reference. These logarithmic scales are widely used in radio frequency and audio engineering to manage the vast dynamic range of signal powers encountered in practical systems, making it easier to compare strengths without dealing with unwieldy large or small numbers.

Power level in dBm to watts — `dbm_to_watts`

The power in watts of a level given in dBm (and the same level in dBW).

$$P = 1\,\mathrm{mW} \cdot 10^{P_{\mathrm{dBm}}/10}$$

symbol		meaning	SI unit
`dBm`	in	power level in dBm	—
`P`	out	power	W
`dBW`	out	power level in dBW	—

Applies when: finite level.

```
$ signal-processing solve dbm_to_watts dBm=30
P = 1 W
dBW = 0
  by: Power level in dBm to watts  (P = 1 mW * 10^(L/10))
  dBm = 30
  assumes: finite level
```

What the worked example means: This calculation converts a signal power level given in decibels relative to one milliwatt into its equivalent linear value in watts. Applying the formula that raises ten to the power of the input divided by ten and multiplying by one milliwatt gives the actual physical power delivered. For an input of 30, the resulting power is 1 W, which also translates to a level of 0 when referenced against one watt.

Power ratio to decibels — `power_ratio_to_db`

A power ratio expressed in decibels, 10*log10(P2/P1).

$$L = 10 \log_{10} \frac{P_2}{P_1}\ \mathrm{dB}$$

symbol		meaning	SI unit
`ratio`	in	power ratio P2/P1 (a positive number)	—
`dB`	out	level in decibels (dB)	—

Applies when: ratio > 0.

```
$ signal-processing solve power_ratio_to_db ratio=200
dB = 23.01029996
  by: Power ratio to decibels  (10 log10(ratio))
  ratio = 200
  assumes: ratio > 0
```

What the worked example means: This calculation converts the power ratio 200 into decibels using the formula ten times the base-ten logarithm of the ratio. The resulting value is 23.0103, which represents the relative power level in a compact logarithmic scale suitable for signal processing applications.

Signal-to-noise ratio from signal and noise powers — `snr_from_powers`

The signal-to-noise ratio Ps/Pn, as a ratio and in dB.

$$\mathrm{SNR_{dB}} = 10 \log_{10} \frac{P_s}{P_n}$$

symbol		meaning	SI unit
`Ps`	in	signal power	W
`Pn`	in	noise power	W
`snr_db`	out	signal-to-noise ratio in dB	—
`snr`	out	signal-to-noise ratio (linear)	—

Applies when: Ps > 0; Pn > 0.

```
$ signal-processing solve snr_from_powers "Ps=2 mW" "Pn=10 uW"
snr = 200
snr_db = 23.01029996
  by: Signal-to-noise ratio from signal and noise powers  (SNR = Ps/Pn; 10 log10(SNR))
  Ps = 0.002 W
  Pn = 1e-05 W
  assumes: Ps > 0; Pn > 0
```

What the worked example means: This calculation determines the quality of a signal by comparing its power against the background noise power. Using the provided signal power of 2 mW and noise power of 10 uW gives the linear ratio 200. This value is then converted to decibels to yield 23.0103, offering a standard logarithmic measure of link clarity where higher values indicate a stronger signal relative to noise.

Total power of several signals given in dBm — `dbm_power_sum`

The combined power of uncorrelated signals whose levels are given in dBm (powers add, levels do not).

$$P_{\mathrm{tot}} = 10 \log_{10} \sum_i 10^{P_i/10} \ \mathrm{dBm}$$

symbol		meaning	SI unit
`levels`	in	power levels in dBm	—
`total_dBm`	out	total power level in dBm	—
`total_W`	out	total power	W

Applies when: uncorrelated signals (their powers add).

```
$ signal-processing solve dbm_power_sum "levels=[10, 10]"
total_dBm = 13.01029996
total_W = 0.02 W
  by: Total power of several signals given in dBm  (convert to milliwatts, add, convert back)
  assumes: uncorrelated signals (their powers add)
```

What the worked example means: This calculation determines the combined power level of multiple uncorrelated signals by first converting their individual levels from decibels relative to one milliwatt into linear power values in milliwatts. These linear powers are then summed together before being converted back into a single logarithmic value. The resulting total level is 13.0103, representing a physical power of 0.02 W. This

approach is necessary because logarithmic units cannot be added directly; only the underlying linear power quantities are additive for independent sources.

Digital communication (5)

Bit and symbol error rates of QPSK in AWGN — `qpsk_error_rates`

Gray-coded coherent QPSK: bit error rate Q(sqrt(2 Eb/N0)) and symbol error rate 2Q - Q^2.

$$P_b = Q\left(\sqrt{2E_b/N_0}\right), \quad P_s = 2Q - Q^2$$

symbol		meaning	SI unit
`ebn0_db`	in	Eb/N0 in dB	—
`ber`	out	bit error rate (Gray coding)	—
`ser`	out	symbol error rate	—

Applies when: coherent detection; Gray coding; AWGN channel.

```
$ signal-processing solve qpsk_error_rates ebn0_db=6
ber = 0.002388290781
ser = 0.004770877629
  by: Bit and symbol error rates of QPSK in AWGN  (independent BPSK on I and Q: P_s = 1 - (1 -
Q)^2)
  ebn0_db = 6
  assumes: coherent detection; Gray coding; AWGN channel
```

What the worked example means: This calculation determines the performance of Gray-coded coherent Quadrature Phase Shift Keying in an Additive White Gaussian Noise channel. By treating the signal as two independent Binary Phase Shift Keying streams, it derives the bit error rate from the input energy per bit to noise power spectral density ratio of 6. The resulting bit error rate is 0.00238829, while the probability that any given symbol is received incorrectly is 0.00477088. These metrics indicate how reliably data can be transmitted under these specific conditions.

Bit error rate of BPSK (and coherent BFSK) in AWGN — `bpsk_ber`

The bit error probability of coherent BPSK, Q(sqrt(2 Eb/N0)), and of coherent orthogonal BFSK, Q(sqrt(Eb/N0)), in additive white Gaussian noise.

$$P_b = Q\left(\sqrt{2E_b/N_0}\right) = \tfrac{1}{2}\operatorname{erfc}\sqrt{E_b/N_0}$$

symbol		meaning	SI unit
`ebn0_db`	in	Eb/N0 in dB	—
`ber`	out	BPSK bit error rate	—
`ber_bfsk`	out	coherent BFSK bit error rate	—

Applies when: coherent detection; AWGN channel; Eb/N0 stated in dB.

```
$ signal-processing solve bpsk_ber ebn0_db=8
ber = 0.0001909077741
ber_bfsk = 0.0060043864
  by: Bit error rate of BPSK (and coherent BFSK) in AWGN  (P_b = Q(sqrt(2 Eb/N0)) =
erfc(sqrt(Eb/N0))/2)
  ebn0_db = 8
  assumes: coherent detection; AWGN channel; Eb/N0 stated in dB
```

What the worked example means: This calculation determines the probability of a bit being received incorrectly for Binary Phase Shift Keying and coherent Binary Frequency Shift Keying. It uses the energy per bit to noise power spectral density ratio, given as 8, to compute these values under additive white Gaussian noise conditions. The resulting BPSK bit error rate of 0.000190908 indicates that phase shift keying is more robust against noise than frequency shift keying, which has a higher error rate of 0.00600439 for the same signal strength.

Bit error rates of DPSK and noncoherent BFSK — `noncoherent_ber`

The bit error rates of differentially coherent BPSK (DPSK), exp(-Eb/N0)/2, and of noncoherent orthogonal BFSK, exp(-Eb/2N0)/2, in AWGN.

$$P_b^{\mathrm{DPSK}} = \tfrac{1}{2} e^{-E_b/N_0}, \quad P_b^{\mathrm{NCFSK}} = \tfrac{1}{2} e^{-E_b/2N_0}$$

symbol		meaning	SI unit
`ebn0_db`	in	Eb/N0 in dB	—
`ber_dpsk`	out	DPSK bit error rate	—
`ber_ncfsk`	out	noncoherent BFSK bit error rate	—

Applies when: AWGN channel; slow phase variation (DPSK); orthogonal tones (FSK).

```
$ signal-processing solve noncoherent_ber ebn0_db=8
ber_dpsk = 0.0009094044481
ber_ncfsk = 0.02132374789
  by: Bit error rates of DPSK and noncoherent BFSK  (closed forms of noncoherent detection)
  ebn0_db = 8
  assumes: AWGN channel; slow phase variation (DPSK); orthogonal tones (FSK)
```

What the worked example means: This calculation determines the probability of bit errors for Differential Phase Shift Keying and noncoherent Binary Frequency Shift Keying systems operating in additive white Gaussian noise. Given a signal-to-noise ratio per bit of 8 decibels, the resulting error rates are 0.000909404 for DPSK and 0.0213237 for noncoherent BFSK. These values indicate how often bits will be incorrectly detected without requiring phase synchronization at the receiver.

Symbol error rate of M-PSK in AWGN — `mpsk_symbol_error`

The exact symbol error rate of coherent M-PSK (M = 2 to 64) at a given Eb/N0 (Craig's integral), and the Gray-coded bit error rate approximation.

$$P_s = \frac{1}{\pi} \int_0^{(M-1)\pi/M} \exp\left(-\frac{E_s}{N_0} \frac{\sin^2(\pi/M)}{\sin^2\theta} \right) d\theta$$

symbol		meaning	SI unit
`M`	in	number of phases	—
`ebn0_db`	in	Eb/N0 in dB	—
`ser`	out	symbol error rate	—
`ber_approx`	out	bit error rate approximation SER/log2 M	—

Applies when: coherent detection; AWGN channel.

```
$ signal-processing solve mpsk_symbol_error M=8 ebn0_db=10
ser = 0.003034185962
ber_approx = 0.001011395321
  by: Symbol error rate of M-PSK in AWGN  (Craig's integral evaluated to 30 digits)
  ebn0_db = 10
  M = 8
  assumes: coherent detection; AWGN channel
```

What the worked example means: This calculation determines the exact symbol error rate for coherent M-PSK modulation over an additive white Gaussian noise channel using Craig's integral. Given a constellation size of 8 and an energy per bit to noise power spectral density ratio of 10, the resulting symbol error rate is 0.00303419. The corresponding bit error rate, approximated by dividing the symbol error rate by the number of bits per symbol, is 0.0010114. These values indicate the probability of detecting the wrong phase or bit under ideal coherent detection conditions.

Symbol error rate of square M-QAM in AWGN — `mqam_symbol_error`

The exact symbol error rate of coherent square M-QAM (4 to 1024 points) at a given Eb/N0, and the Gray-coded bit error rate approximation SER/log2 M.

$$P_s = 1 - \left[1 - 2\left(1 - \frac{1}{\sqrt{M}}\right) Q\left(\sqrt{\frac{3E_s}{(M-1)N_0}}\right)\right]^2$$

symbol		meaning	SI unit
`M`	in	constellation size	—
`ebn0_db`	in	Eb/N0 in dB	—
`ser`	out	symbol error rate	—
`ber_approx`	out	bit error rate approximation (Gray coding)	—

Applies when: square constellation; coherent detection; AWGN channel.

```
$ signal-processing solve mqam_symbol_error M=16 ebn0_db=10
ser = 0.007004294294
ber_approx = 0.001751073574
  by: Symbol error rate of square M-QAM in AWGN  (P_s = 1 - (1 - P_L)^2 with P_L = 2(1 - 1/sqrt(M))
Q(sqrt(3 Es / ((M-1) N0))))
  ebn0_db = 10
  M = 16
  assumes: square constellation; coherent detection; AWGN channel
```

What the worked example means: This calculation determines the exact symbol error rate for a square M-QAM constellation operating over an additive white Gaussian noise channel. Using the specified constellation size of 16 and an energy per bit to noise power spectral density ratio of 10, the model computes a symbol error rate of 0.00700429. It also provides a bit error rate approximation of 0.00175107, which assumes Gray coding maps adjacent symbols to bit strings differing by only one bit.

Digital filters (10)

Bilinear transform of a first-order low-pass filter — `bilinear_rc_lowpass`

The digital filter H(z) = (b0 + b1 z^-1)/(1 + a1 z^-1) obtained from H(s) = wc/(s + wc) by the bilinear transform s = 2 fs (1 - z^-1)/(1 + z^-1), without pre-warping.

$$H(z) = \frac{\omega_c(1 + z^{-1})}{(\omega_c + 2f_s) + (\omega_c - 2f_s)z^{-1}}$$

symbol		meaning	SI unit
`fc`	in	analog cutoff frequency	Hz
`fs`	in	sampling rate	Hz
`b0`	out	numerator coefficient b0	—
`b1`	out	numerator coefficient b1	—
`a1`	out	denominator coefficient a1 (a0 = 1)	—

Applies when: fc > 0; fs > 0; no frequency pre-warping.

```
$ signal-processing solve bilinear_rc_lowpass "fs=8 kHz" "fc=1 kHz"
b0 = 0.2819698001
b1 = 0.2819698001
a1 = -0.4360603998
  by: Bilinear transform of a first-order low-pass filter  (substitute s = 2 fs (1 - z^-1)/(1 +
z^-1) and normalize a0 = 1)
  fs = 8000 Hz
  fc = 1000 Hz
  assumes: fc > 0; fs > 0; no frequency pre-warping
```

What the worked example means: This calculation determines the numerator and denominator coefficients for a first-order digital low-pass filter derived from an analog prototype using the bilinear transform. Substituting the sampling relationship into the continuous transfer function gives the specific discrete-time values b0 = 0.28197, b1 = 0.28197, and a1 = -0.43606 based on the cutoff frequency of 1 kHz and the sampling rate of 8 kHz. These coefficients define how past input and output samples are weighted to produce the current filtered signal.

DC gain of a digital filter — `dc_gain`

The gain H(1) = sum b / sum a of the filter at zero frequency, and its level in dB.

$$H(1) = \frac{\sum_k b_k}{\sum_k a_k}$$

symbol		meaning	SI unit
`b`	in	numerator coefficients b	—
`a`	in	denominator coefficients a (1 if not given) (optional)	—
`gain`	out	DC gain H(z = 1) (signed)	—
`gain_db`	out	20 log10 |H(1)| in dB	—

Applies when: sum of a != 0 (no pole at z = 1).

```
$ signal-processing solve dc_gain "b=[1, 2, 1]" "a=[1, -0.5, 0.25]"
gain = 5.333333333
gain_db = 14.53997456
  by: DC gain of a digital filter  (H(1) = sum b_k / sum a_k)
  assumes: sum of a != 0 (no pole at z = 1)
```

What the worked example means: This calculation determines the steady-state amplification or attenuation of a constant input signal by evaluating the filter's transfer function at zero frequency. The result, 5.33333, represents the ratio of the sum of numerator coefficients to the sum of denominator coefficients. This value indicates how much a direct current component is scaled by the system, while 14.54 expresses this scaling factor on a logarithmic decibel scale for easier interpretation of dynamic range.

FIR filter output — `fir_filter_output`

The output y[n] = sum_k b[k] x[n-k] of an FIR filter with coefficients b (zero initial conditions, same length as x).

$$y[n] = \sum_{k=0}^{M} b_k \, x[n-k]$$

symbol		meaning	SI unit
`b`	in	filter coefficients (taps) b[0..M]	—
`x`	in	input samples	—
`y`	out	output samples	—

Applies when: zero initial conditions.

```
$ signal-processing solve fir_filter_output "b=[0.5, 0.5]" "x=[1, 2, 3, 4]"
y = [0.5,1.5,2.5,3.5]
  by: FIR filter output  (direct-form FIR difference equation)
  assumes: zero initial conditions
```

What the worked example means: This calculation determines the time-domain response of a finite impulse response filter. It computes each output sample [0.5, 1.5, 2.5, 3.5] by summing the products of filter coefficients [0.5, 0.5] and corresponding delayed input samples [1, 2, 3, 4]. This process shapes the signal spectrum without using feedback, ensuring inherent stability and allowing for precise control over frequency characteristics.

Frequency response of a digital filter at one frequency — `frequency_response`

H(e^{jw}) = B(e^{jw}) / A(e^{jw}) at w = 2 pi f / fs: magnitude, magnitude in dB and phase.

$$H(e^{j\omega}) = \frac{\sum_k b_k e^{-j\omega k}}{\sum_k a_k e^{-j\omega k}},\ \omega = 2\pi f / f_s$$

symbol		meaning	SI unit
`b`	in	numerator coefficients b	—
`f`	in	frequency	Hz
`fs`	in	sampling rate	Hz
`a`	in	denominator coefficients a (1 if not given) (optional)	—
`magnitude`	out	|H|	—
`magnitude_db`	out	20 log10 |H| in dB	—
`phase`	out	phase of H in radians	rad

Applies when: 0 <= f <= fs/2; no pole on the unit circle at f.

```
$ signal-processing solve frequency_response "b=[1, 1]" "fs=8 kHz" "f=1 kHz"
magnitude = 1.847759065
magnitude_db = 5.332906832
phase = -0.3926990817 rad
  by: Frequency response of a digital filter at one frequency  (evaluation of the numerator and
denominator polynomials in z^-1 on the unit circle)
  fs = 8000 Hz
  f = 1000 Hz
  assumes: 0 <= f <= fs/2; no pole on the unit circle at f
```

What the worked example means: The capability evaluates the digital filter's transfer function at the specified frequency by substituting complex exponentials for delay operators. Using the numerator and denominator coefficients, it computes the resulting response. At input frequency 1 kHz with sampling rate 8 kHz, the output shows a linear magnitude of 1.84776, which corresponds to 5.33291 in decibels, and a phase shift of -0.392699 rad radians.

Group delay of a digital filter — `group_delay`

The group delay -d(phase)/dw of H(z) = B(z)/A(z) at frequency f, in samples and in seconds.

$$\tau_g(\omega) = -\frac{d\angle H(e^{j\omega})}{d\omega} = \mathrm{Re}\frac{\sum k b_k e^{-j\omega k}}{\sum b_k e^{-j\omega k}} - \mathrm{Re}\frac{\sum k a_k e^{-j\omega k}}{\sum a_k e^{-j\omega k}}$$

symbol		meaning	SI unit
`b`	in	numerator coefficients b	—
`f`	in	frequency	Hz
`fs`	in	sampling rate	Hz
`a`	in	denominator coefficients a (1 if not given) (optional)	—
`samples`	out	group delay in samples	—
`seconds`	out	group delay in seconds	s

Applies when: 0 <= f <= fs/2; no zero or pole of H on the unit circle at f.

```
$ signal-processing solve group_delay "b=[1, 2, 3, 2, 1]" "fs=8 kHz" "f=500 Hz"
samples = 2
seconds = 0.00025 s
  by: Group delay of a digital filter  (tau = Re[sum k b_k z^-k / sum b_k z^-k] - Re[the same for
a] on the unit circle)
  fs = 8000 Hz
  f = 500 Hz
  assumes: 0 <= f <= fs/2; no zero or pole of H on the unit circle at f
```

What the worked example means: This calculation determines the group delay of a digital filter at the specified frequency. Evaluating the real part of the ratio of coefficient-weighted sums on the unit circle shows that the signal experiences a delay of 2 samples. When converted using the sampling rate 8 kHz, this corresponds to a time delay of 0.00025 s seconds, representing how much the envelope of the signal component at 500 Hz is shifted in time.

IIR filter output (difference equation) — `iir_filter_output`

The output of the recursive filter a[0] y[n] = sum_k b[k] x[n-k] - sum_{k>=1} a[k] y[n-k] (zero initial conditions).

$$a_0 y[n] = \sum_k b_k x[n-k] - \sum_{k \geq 1} a_k y[n-k]$$

symbol		meaning	SI unit
`b`	in	feed-forward coefficients b	—
`a`	in	feedback coefficients a (a[0] != 0)	—
`x`	in	input samples	—
`y`	out	output samples	—

Applies when: a[0] != 0; zero initial conditions.

```
$ signal-processing solve iir_filter_output "b=[1]" "a=[1, -0.5]" "x=[1, 0, 0, 0]"
y = [1.0,0.5,0.25,0.125]
  by: IIR filter output (difference equation)  (direct-form I difference equation)
  assumes: a[0] != 0; zero initial conditions
```

What the worked example means: This calculation determines the time-domain response of an infinite impulse response filter using the direct-form difference equation. The process combines current and past input samples weighted by feed-forward coefficients with past output samples weighted by feedback coefficients to produce the final sequence. For the given inputs, the resulting output values are [1, 0.5, 0.25, 0.125], representing the filtered signal under zero initial conditions.

Impulse response of a digital filter — `impulse_response`

The first n samples of the impulse response h[n] of the filter with coefficients b and a.

$$h[n] = \mathcal{Z}^{-1}\{B(z)/A(z)\}$$

symbol		meaning	SI unit
`b`	in	numerator coefficients b	—
`a`	in	denominator coefficients a (a[0] != 0)	—
`n`	in	number of samples	—
`h`	out	impulse response h[0..n-1]	—

Applies when: a[0] != 0.

```
$ signal-processing solve impulse_response "b=[1]" "a=[1, -0.5]" n=5
h = [1.0,0.5,0.25,0.125,0.0625]
  by: Impulse response of a digital filter  (the difference equation driven by a unit impulse)
  assumes: a[0] != 0
```

What the worked example means: This calculation determines the impulse response h of a digital filter defined by numerator coefficients b and denominator coefficients a. Driving the difference equation with a unit impulse gives the first n samples of the system's characteristic signature. The result, [1, 0.5, 0.25, 0.125, 0.0625], represents how the filter reacts to an instantaneous spike at time zero, which is fundamental for predicting its behavior on any arbitrary input through convolution.

Minimum order of a digital (bilinear) Butterworth filter — `butterworth_order_digital`

The smallest order meeting a passband edge with at most Rp dB attenuation and a stopband edge with at least As dB (digital design by the bilinear transform: edges pre-warped with tan(pi f / fs)).

$$N \geq \frac{\log_{10} \frac{10^{A_s/10}-1}{10^{R_p/10}-1}}{2\log_{10}(\omega_s/\omega_p)}$$

symbol		meaning	SI unit
`fp`	in	passband edge frequency	Hz
`fstop`	in	stopband edge frequency	Hz
`fs`	in	sampling rate	Hz
`Rp`	in	maximum passband attenuation (ripple) in dB	—
`As`	in	minimum stopband attenuation in dB	—
`order`	out	minimum filter order	—
`type`	out	lowpass (fp < fstop) or highpass (fp > fstop)	—

Applies when: 0 < Rp < As; fp != fstop; both edges below fs/2.

```
$ signal-processing solve butterworth_order_digital "fp=1 kHz" "fstop=2 kHz" "fs=10 kHz" Rp=1 As=40
order = 7
type = lowpass
  by: Minimum order of a digital (bilinear) Butterworth filter  (N = ceil(
log10((10^(As/10)-1)/(10^(Rp/10)-1)) / (2 log10(ws/wp)) ), with wp = tan(pi fp/fs), ws = tan(pi
fstop/fs))
  fp = 1000 Hz
  fstop = 2000 Hz
  fs = 10000 Hz
  Rp = 1
  As = 40
  assumes: 0 < Rp < As; fp != fstop; both edges below fs/2
```

What the worked example means: This calculation determines the smallest integer order required for a digital Butterworth filter to meet specific attenuation constraints. Using the bilinear transform method, the passband edge frequency of 1 kHz and stopband edge frequency of 2 kHz are pre-warped relative to the sampling rate of 10 kHz. The resulting minimum order is 7, ensuring that the filter provides at most 1 dB attenuation in the passband and at least 40 dB attenuation in the stopband.

Moving-average filter output — `moving_average`

The causal M-point moving average y[n] = (1/M) sum_{i=0}^{M-1} x[n-i], with x[n] = 0 before the first sample.

$$y[n] = \frac{1}{M} \sum_{i=0}^{M-1} x[n-i]$$

symbol		meaning	SI unit
`x`	in	input samples	—
`M`	in	window length (points averaged)	—
`y`	out	output samples (same length as x)	—

Applies when: M >= 1; zero initial conditions.

```
$ signal-processing solve moving_average M=3 "x=[3, 6, 9, 12]"
y = [1.0,3.0,6.0,9.0]
  by: Moving-average filter output  (direct sum of the last M samples divided by M)
  assumes: M >= 1; zero initial conditions
```

What the worked example means: This calculation produces a smoothed signal by averaging a fixed window of recent data points. The result [1, 3, 6, 9] represents the local mean of the input sequence [3, 6, 9, 12] over the specified window length M. This operation effectively reduces high-frequency noise while preserving the general trend of the underlying signal, acting as a simple low-pass filter.

Step response of a digital filter — `step_response`

The first n samples of the response s[n] of the filter (b, a) to a unit step.

$$s[n] = \sum_{m=0}^{n} h[m]$$

symbol		meaning	SI unit
`b`	in	numerator coefficients b	—
`a`	in	denominator coefficients a (a[0] != 0)	—
`n`	in	number of samples	—
`s`	out	step response s[0..n-1]	—

Applies when: a[0] != 0.

```
$ signal-processing solve step_response "b=[1]" "a=[1, -0.5]" n=4
s = [1.0,1.5,1.75,1.875]
  by: Step response of a digital filter  (the difference equation driven by a unit step)
  assumes: a[0] != 0
```

What the worked example means: This calculation determines the time-domain behavior of a digital filter when subjected to an abrupt change in input level. Applying a unit step signal to the system characterized by numerator coefficients [1] and denominator coefficients [1, -0.5] gives the output sequence [1, 1.5, 1.75, 1.875] spanning 4 samples. The resulting values represent how the filter transitions from rest to its steady state, revealing critical characteristics such as rise time, overshoot, and settling speed which are essential for evaluating transient performance in control and communication applications.

Filter design (3)

Digital Butterworth filter design (bilinear transform) — `butterworth_design`

The coefficients b and a of an N-th order digital Butterworth low-pass or high-pass filter with cutoff fc at sampling rate fs (bilinear transform with pre-warping).

$$s = 2f_s \frac{1 - z^{-1}}{1 + z^{-1}}, \quad \omega_c = 2f_s \tan \frac{\pi f_c}{f_s}$$

symbol		meaning	SI unit
`N`	in	filter order (1..8)	—
`fc`	in	cutoff (-3 dB) frequency	Hz
`fs`	in	sampling rate	Hz
`btype`	in	lowpass or highpass	—
`b`	out	numerator coefficients b[0..N]	—
`a`	out	denominator coefficients a[0..N] (a[0] = 1)	—

Applies when: 0 < fc < fs/2; 1 <= N <= 8.

```
$ signal-processing solve butterworth_design btype=lowpass N=2 "fs=8 kHz" "fc=1 kHz"
b = [0.0976310729378,0.195262145876,0.0976310729378]
a = [1.0,-0.942809041582,0.333333333333]
  by: Digital Butterworth filter design (bilinear transform)  (analog prototype poles, frequency
scaling with the pre-warped cutoff 2 fs tan(pi fc / fs), bilinear transform, polynomial expansion)
  fs = 8000 Hz
  fc = 1000 Hz
  btype = lowpass
  assumes: 0 < fc < fs/2; 1 <= N <= 8
```

What the worked example means: This calculation determines the digital filter coefficients by mapping an analog prototype to the discrete domain using the bilinear transform. The process begins with pre-warping the cutoff frequency 1 kHz based on the sampling rate 8 kHz, resulting in a warped angular frequency. Analog poles are then calculated for the specified order 2 and type lowpass. These poles are transformed into the z-plane, and the resulting system function is expanded into polynomial form. The final output provides the numerator coefficients [0.0976311, 0.195262, 0.0976311] and denominator coefficients [1, -0.942809, 0.333333], which define the difference equation for implementation. This ensures a maximally flat passband response while maintaining stability within the digital framework.

FIR low-pass filter design by the window method — `fir_window_design`

The N taps of a linear-phase FIR low-pass filter with cutoff fc: an ideal (sinc) impulse response shifted by (N-1)/2, multiplied by a rectangular, Hann, Hamming or Blackman window and scaled to unit DC gain.

$$h[n] \propto \frac{2f_c}{f_s}\,\mathrm{sinc}\!\left(\frac{2f_c}{f_s}\left(n-\tfrac{N-1}{2}\right)\right)w[n]$$

symbol		meaning	SI unit
`N`	in	number of taps	—
`fc`	in	cutoff frequency	Hz
`fs`	in	sampling rate	Hz
`window`	in	window type	—
`h`	out	filter taps h[0..N-1]	—

Applies when: 0 < fc < fs/2; symmetric window; taps scaled so that the DC gain is 1.

```
$ signal-processing solve fir_window_design window=hamming N=5 "fs=8 kHz" "fc=1 kHz"
h = [0.024553834015,0.234389464238,0.482113403494,0.234389464238,0.024553834015]
  by: FIR low-pass filter design by the window method  (windowed ideal low-pass: h[n] = (2fc/fs)
sinc(2fc/fs (n - (N-1)/2)) w[n], normalized to sum 1)
  fs = 8000 Hz
  fc = 1000 Hz
  window = hamming
  assumes: 0 < fc < fs/2; symmetric window; taps scaled so that the DC gain is 1
```

What the worked example means: This calculation produces the impulse response coefficients for a linear-phase finite impulse response low-pass filter. The process starts with an ideal sinc function defined by the cutoff frequency 1 kHz and sampling rate 8 kHz, which is then multiplied by a symmetric window to reduce side lobes. The resulting taps, denoted as [0.0245538, 0.234389, 0.482113, 0.234389, 0.0245538], are

normalized so that the filter passes direct current signals without attenuation while rejecting frequencies above the specified cutoff.

Second-order IIR notch filter design — `notch_filter_design`

The coefficients of a second-order IIR notch filter removing frequency f0 with quality factor Q (bandwidth f0/Q at -3 dB) at sampling rate fs.

$$H(z) = g\,\frac{1 - 2\cos\omega_0 z^{-1} + z^{-2}}{1 - 2g\cos\omega_0 z^{-1} + (2g-1)z^{-2}},\; g = \frac{1}{1+\tan(\omega_0/2Q)}$$

symbol		meaning	SI unit
`f0`	in	frequency to remove	Hz
`Q`	in	quality factor f0 / bandwidth	—
`fs`	in	sampling rate	Hz
`b`	out	numerator coefficients b0, b1, b2	—
`a`	out	denominator coefficients 1, a1, a2	—

Applies when: 0 < f0 < fs/2; Q > 0.

```
$ signal-processing solve notch_filter_design "fs=1 kHz" "f0=60 Hz" Q=30
b = [0.9937559649536571,-1.8479418578501994,0.9937559649536571]
a = [1.0,-1.8479418578501994,0.9875119299073143]
  by: Second-order IIR notch filter design  (zeros on the unit circle at f0; poles at the same
angle, radius set by tan(w0 / 2Q))
  fs = 1000 Hz
  f0 = 60 Hz
  Q = 30
  assumes: 0 < f0 < fs/2; Q > 0
```

What the worked example means: This calculation determines the numerator and denominator coefficients for a second-order IIR notch filter. Using the target frequency 60 Hz, quality factor 30, and sampling rate 1 kHz, it computes the gain parameter g to place zeros on the unit circle at the desired notch location while setting pole radii to control bandwidth. The resulting output coefficients [0.993756, -1.84794, 0.993756] and [1, -1.84794, 0.987512] define the difference equation that attenuates signals near 60 Hz by approximately three decibels within the specified bandwidth.

Fourier series and transforms (5)

Analytic signal, Hilbert transform and envelope — `analytic_envelope`

The discrete Hilbert transform of a real sequence and its envelope |x + j H{x}|, computed through the DFT (negative frequencies removed, positive ones doubled).

$$x_a[n] = \mathrm{IDFT}\{X[k]\,h[k]\},\quad \text{envelope} = |x_a[n]|$$

symbol		meaning	SI unit
`x`	in	real samples	—

symbol		meaning	SI unit
`envelope`	out	envelope |x_a[n]|	—
`hilbert`	out	Hilbert transform (imaginary part of the analytic signal)	—

Applies when: the sequence treated as one period (DFT-based transform).

```
$ signal-processing solve analytic_envelope "x=[1, 0, -1, 0]"
envelope = [1.0,1.0,1.0,1.0]
hilbert = [0.0,1.0,0.0,-1.0]
  by: Analytic signal, Hilbert transform and envelope  (FFT, multiply by h[k] = 1, 2, ..., 2, (1),
0, ..., 0, inverse FFT)
  assumes: the sequence treated as one period (DFT-based transform)
```

What the worked example means: This calculation derives the analytic signal from the real input samples [1, 0, -1, 0] by applying a discrete Hilbert transform via frequency domain manipulation. The resulting envelope, representing the instantaneous amplitude, is [1, 1, 1, 1], while the imaginary part corresponding to the Hilbert transform is [0, 1, 0, -1]. This process effectively removes negative frequencies and doubles positive ones to create a complex representation suitable for modulation analysis.

Fourier transform of a decaying exponential — `exponential_pulse_spectrum`

X(f) = A tau / (1 + j 2 pi f tau) of x(t) = A exp(-t/tau) for t >= 0: magnitude, phase, real and imaginary parts.

$$X(f) = \frac{A\tau}{1 + j2\pi f\tau}$$

symbol		meaning	SI unit
`A`	in	initial amplitude	—
`tau`	in	time constant	s
`f`	in	frequency	Hz
`magnitude`	out	|X(f)|	—
`phase`	out	phase of X(f) in radians	rad
`re`	out	real part	—
`im`	out	imaginary part	—

Applies when: A > 0; tau > 0; x(t) = 0 for t < 0.

```
$ signal-processing solve exponential_pulse_spectrum "tau=0.5 s" "f=0 Hz" A=2
magnitude = 1
phase = 0 rad
re = 1
im = 0
  by: Fourier transform of a decaying exponential  (X(f) = integral_0^inf A e^{-t/tau} e^{-j 2 pi f
t} dt in closed form)
  tau = 0.5 s
  f = 0 Hz
  A = 2
  assumes: A > 0; tau > 0; x(t) = 0 for t < 0
```

What the worked example means: This calculation determines the frequency-domain representation of a causal signal defined by an initial amplitude 2 and time constant 0.5 s at frequency 0 Hz. The resulting complex spectrum has a real part of 1, an imaginary part of 0, a total magnitude of 1, and a phase angle of 0 rad. These values describe how the signal's energy is distributed across frequencies, revealing its low-pass filtering characteristics.

Fourier transform of a rectangular pulse — `rect_pulse_spectrum`

X(f) = A tau sinc(f tau) of a rectangular pulse of amplitude A and width tau centred at t = 0, its magnitude, and the first spectral null 1/tau.

$$X(f) = A\tau\,\mathrm{sinc}(f\tau) = A\tau\frac{\sin(\pi f\tau)}{\pi f\tau}$$

symbol		meaning	SI unit
`A`	in	pulse amplitude	—
`tau`	in	pulse width	s
`f`	in	frequency	Hz
`X`	out	X(f) (real, in amplitude-seconds)	—
`magnitude`	out	|X(f)|	—
`first_null`	out	first zero of the spectrum, 1/tau	Hz

Applies when: A != 0; tau > 0; the pulse centred at t = 0.

```
$ signal-processing solve rect_pulse_spectrum "tau=1 ms" "f=250 Hz" A=2
X = 0.001800632632
magnitude = 0.001800632632
first_null = 1000 Hz
  by: Fourier transform of a rectangular pulse  (X(f) = A tau sin(pi f tau)/(pi f tau))
  tau = 0.001 s
  f = 250 Hz
  A = 2
  assumes: A != 0; tau > 0; the pulse centred at t = 0
```

What the worked example means: This calculation determines the continuous-time Fourier transform of a rectangular pulse defined by amplitude 2 and width 1 ms. The resulting spectrum, evaluated at frequency 250 Hz, is 0.00180063, which represents the signal's component in the frequency domain. The absolute strength of this component is given by the magnitude 0.00180063. Additionally, the analysis identifies the first frequency where the spectrum drops to zero as 1000 Hz, illustrating the inverse relationship between the pulse duration in time and its bandwidth.

Harmonic amplitude of a square, triangle or sawtooth wave — `fourier_series_harmonic`

The amplitude of the n-th harmonic in the Fourier series of a zero-mean square, triangle or sawtooth wave of peak amplitude A, relative to the fundamental, and its RMS value.

$$c_n^{\mathrm{sq}} = \frac{4A}{n\pi},\; c_n^{\mathrm{tri}} = \frac{8A}{\pi^2 n^2}\;(n\ \mathrm{odd}),\quad c_n^{\mathrm{saw}} = \frac{2A}{n\pi}$$

symbol		meaning	SI unit
`waveform`	in	waveform	—
`A`	in	peak amplitude of the wave	—
`n`	in	harmonic number (1 = fundamental)	—
`amplitude`	out	amplitude of the n-th harmonic	—
`relative`	out	amplitude relative to the fundamental	—
`rms`	out	RMS value of the n-th harmonic	—

Applies when: zero-mean waveform with peak A; 50 % duty square wave; symmetric triangle; linear sawtooth.

```
$ signal-processing solve fourier_series_harmonic waveform=square n=3 A=2
amplitude = 0.8488263632
relative = 0.3333333333
rms = 0.6002108774
  by: Harmonic amplitude of a square, triangle or sawtooth wave  (Fourier coefficients: square
4A/(n pi) (odd n), triangle 8A/(pi^2 n^2) (odd n), sawtooth 2A/(n pi))
  A = 2
  waveform = square
  assumes: zero-mean waveform with peak A; 50 % duty square wave; symmetric triangle; linear
sawtooth
```

What the worked example means: This calculation determines the spectral content of a periodic waveform by finding the amplitude of its specific harmonics. For a zero-mean square, triangle, or sawtooth wave with peak amplitude 2, the amplitude of the 3-th harmonic is computed as 0.848826. This result allows engineers to quantify how much energy resides at that frequency relative to the fundamental component, yielding a ratio of 0.333333 and an effective RMS value of 0.600211, which are critical for assessing signal distortion and filter requirements.

Total harmonic distortion — `thd`

THD = sqrt(V2^2 + V3^2 + ...)/V1 from the amplitudes of the fundamental and its harmonics, as a ratio, in percent and in dB.

$$\mathrm{THD} = \frac{\sqrt{\sum_{n \geq 2} V_n^2}}{V_1}$$

symbol		meaning	SI unit
`V`	in	amplitudes [V1, V2, V3, ...] of the fundamental and harmonics	—
`thd`	out	THD as a ratio	—
`percent`	out	THD in percent	—
`db`	out	THD in dB	—

Applies when: V1 > 0; amplitudes >= 0 (RMS or peak, all alike).

```
$ signal-processing solve thd "V=[10, 0.5, 0.3, 0.1]"
thd = 0.05916079783
percent = 5.916079783
db = -24.55931956
  by: Total harmonic distortion  (root-sum-square of the harmonics over the fundamental)
  assumes: V1 > 0; amplitudes >= 0 (RMS or peak, all alike)
```

What the worked example means: This calculation determines the total harmonic distortion by taking the root-sum-square of all harmonic amplitudes and dividing that value by the fundamental amplitude. The result, expressed as a ratio 0.0591608, indicates the relative strength of unwanted harmonics compared to the primary signal frequency. This metric is also presented in percentage form as 5.91608 and in logarithmic decibel units as -24.5593 for easier interpretation across different dynamic ranges.

Modulation (3)

AM modulation index from the envelope — `am_modulation_index`

The modulation index m = (Vmax - Vmin)/(Vmax + Vmin) of an AM wave from the maximum and minimum of its envelope, and the carrier amplitude.

$$m = \frac{V_{\mathrm{max}} - V_{\mathrm{min}}}{V_{\mathrm{max}} + V_{\mathrm{min}}}$$

symbol		meaning	SI unit
`Vmax`	in	maximum of the envelope	V
`Vmin`	in	minimum of the envelope	V
`m`	out	modulation index	—
`Ac`	out	carrier amplitude (Vmax + Vmin)/2	V

Applies when: 0 <= Vmin < Vmax; sinusoidal message.

```
$ signal-processing solve am_modulation_index "Vmax=10 V" "Vmin=2 V"
m = 0.6666666667
Ac = 6 V
  by: AM modulation index from the envelope  (m = (Vmax - Vmin)/(Vmax + Vmin))
  Vmax = 10 V
  Vmin = 2 V
  assumes: 0 <= Vmin < Vmax; sinusoidal message
```

What the worked example means: This calculation determines the modulation depth of an amplitude modulated wave using the maximum and minimum values observed in its envelope. The modulation index, which indicates how much the carrier signal varies, is found to be 0.666667. Additionally, the carrier amplitude, representing the baseline strength of the unmodulated signal, is calculated as 6 V. These results help assess whether the transmission is properly modulated or if it suffers from under-modulation or over-modulation.

FM spectrum: Carson bandwidth and Bessel sideband amplitudes — `fm_spectrum`

For sinusoidal FM with peak deviation df and modulating frequency fm: the index beta = df/fm, Carson's bandwidth 2(df + fm), the carrier and n-th sideband amplitudes J0(beta), Jn(beta) (relative to the unmodulated

carrier) and the power fraction inside Carson's bandwidth.

$$B_C = 2(\Delta f + f_m), \quad s(t) = A\sum_n J_n(\beta)\cos[2\pi(f_c + nf_m)t]$$

symbol		meaning	SI unit
`df`	in	peak frequency deviation	Hz
`fm`	in	modulating (message) frequency	Hz
`n`	in	sideband number (default 1) (optional)	—
`carson_bandwidth`	out	Carson's rule bandwidth 2(df + fm)	Hz
`beta`	out	modulation index df/fm	—
`carrier`	out	carrier amplitude J0(beta)	—
`Jn`	out	amplitude of each n-th sideband Jn(beta)	—
`pair_power`	out	power fraction in the pair of n-th sidebands	—
`power_in_carson`	out	power fraction within Carson's bandwidth	—

Applies when: sinusoidal message; beta <= 100.

```
$ signal-processing solve fm_spectrum "df=75 kHz" "fm=15 kHz"
carson_bandwidth = 180000 Hz
beta = 5
carrier = -0.1775967713
Jn = -0.3275791376
pair_power = 0.2146161828
power_in_carson = 0.9935588993
  by: FM spectrum: Carson bandwidth and Bessel sideband amplitudes  (Bessel functions of the first
kind (30-digit evaluation); Carson's rule)
  df = 75000 Hz
  fm = 15000 Hz
  assumes: sinusoidal message; beta <= 100
```

What the worked example means: For a peak deviation of 75 kHz Hz and modulating frequency of 15 kHz Hz, the modulation index is 5, Carson's bandwidth is 180000 Hz Hz, and the carrier amplitude is -0.177597.

Power in an AM (DSB full-carrier) signal — `am_power`

Total power Pc (1 + m^2/2), sideband power, power per sideband and transmission efficiency of an AM signal with carrier power Pc and modulation index m (sinusoidal message).

$$P_t = P_c\left(1 + \frac{m^2}{2}\right)$$

symbol		meaning	SI unit
`Pc`	in	carrier power	W
`m`	in	modulation index (0 < m <= 1)	—
`Pt`	out	total transmitted power	W
`Psb`	out	power in both sidebands	W

symbol		meaning	SI unit
`Pusb`	out	power in one sideband	W
`efficiency`	out	fraction of the power in the sidebands	—

Applies when: 0 < m <= 1; a single sinusoidal message.

```
$ signal-processing solve am_power "Pc=1000 W" m=0.8
Pt = 1320 W
Psb = 320 W
Pusb = 160 W
efficiency = 0.2424242424
  by: Power in an AM (DSB full-carrier) signal  (Pt = Pc (1 + m^2/2); each sideband Pc m^2/4)
  Pc = 1000 W
  m = 0.8
  assumes: 0 < m <= 1; a single sinusoidal message
```

What the worked example means: This calculation determines the total transmitted power, sideband powers, and efficiency for an amplitude modulated signal with a full carrier. Given a carrier power of 1000 W and a modulation index of 0.8, the total power is found to be 1320 W. The combined power in both sidebands is 320 W, while the power in a single sideband is 160 W. The transmission efficiency, representing the fraction of power carrying information, is calculated as 0.242424. This result helps engineers assess how effectively the available power is used for transmitting the message rather than just the carrier.

Noise and link budgets (3)

Free-space path loss — `free_space_path_loss`

The loss (4 pi d f / c)^2 between isotropic antennas a distance d apart at frequency f, in dB and as a ratio.

$$L_{\mathrm{fs}} = \left(\frac{4\pi d f}{c}\right)^2$$

symbol		meaning	SI unit
`d`	in	distance	m
`f`	in	frequency	Hz
`loss_db`	out	free-space path loss in dB	—
`loss`	out	path loss as a power ratio	—
`wavelength`	out	wavelength	m

Applies when: free space, line of sight; far field (d >= one wavelength); isotropic antennas.

```
$ signal-processing solve free_space_path_loss "d=100 m" "f=2.4 GHz"
loss_db = 80.05200806
loss = 101204728.8
wavelength = 0.1249135242 m
  by: Free-space path loss  (FSPL = (4 pi d / lambda)^2)
  d = 100 m
  f = 2400000000 Hz
  assumes: free space, line of sight; far field (d >= one wavelength); isotropic antennas
```

What the worked example means: This calculation determines the attenuation of a radio signal traveling through empty space between two isotropic antennas. Using the distance 100 m and frequency 2.4 GHz, the resulting wavelength is 0.124914 m. The computed free-space path loss is 80.052 dB, which represents a power ratio of 1.01205e+08. This value indicates the minimum signal reduction due solely to geometric spreading in an ideal environment.

Noise figure of cascaded stages (Friis formula) — `noise_figure_cascade`

The overall noise figure, noise factor, equivalent noise temperature and gain of a chain of matched stages with given noise figures and gains.

$$F = F_1 + \frac{F_2 - 1}{G_1} + \frac{F_3 - 1}{G_1 G_2} + \cdots$$

symbol		meaning	SI unit
`nf_db`	in	noise figure of each stage in dB, input first	—
`gain_db`	in	available gain of each stage in dB	—
`nf_db`	out	overall noise figure in dB	—
`F`	out	overall noise factor	—
`Te`	out	equivalent input noise temperature	K
`gain_db`	out	overall gain in dB	—

Applies when: impedance-matched stages; noise figures referred to T0 = 290 K; one gain per stage.

```
$ signal-processing solve noise_figure_cascade "nf_db=[2, 6, 10]" "gain_db=[15, 20, 10]"
nf_db = 2.258283171
F = 1.682009007
Te = 197.782612 K
gain_db = 45
  by: Noise figure of cascaded stages (Friis formula)  (Friis: F = F1 + (F2 - 1)/G1 + (F3 - 1)/(G1
G2) + ...)
  assumes: impedance-matched stages; noise figures referred to T0 = 290 K; one gain per stage
```

What the worked example means: This calculation determines the cumulative noise performance of a cascade of impedance-matched stages using the Friis formula. The result, an overall noise figure of 2.25828, reflects how the noise contributions of subsequent stages are attenuated by the gain of preceding stages. A higher gain in the first stage, indicated by [15, 20, 10], generally reduces the impact of later stages on the final noise figure.

Received power of a radio link (Friis transmission equation) — `friis_received_power`

The power received over a free-space link: Pr = Pt Gt Gr (lambda / 4 pi d)^2, in dBm and watts.

$$P_r = P_t G_t G_r \left(\frac{\lambda}{4\pi d} \right)^2$$

symbol		meaning	SI unit
`Pt`	in	transmit power	W
`Gt_db`	in	transmit antenna gain in dBi	—
`Gr_db`	in	receive antenna gain in dBi	—
`f`	in	frequency	Hz
`d`	in	distance	m
`Pr_dbm`	out	received power in dBm	—
`Pr`	out	received power	W

Applies when: free space, line of sight; far field; matched, co-polarized antennas.

```
$ signal-processing solve friis_received_power "Pt=1 W" Gt_db=10 Gr_db=5 "f=2.4 GHz" "d=1 km"
Pr_dbm = -55.05200806
Pr = 3.12463429e-09 W
  by: Received power of a radio link (Friis transmission equation)  (link budget in dB: Pt + Gt +
Gr - FSPL)
  Pt = 1 W
  Gt_db = 10
  Gr_db = 5
  f = 2400000000 Hz
  d = 1000 m
  assumes: free space, line of sight; far field; matched, co-polarized antennas
```

What the worked example means: This calculation determines the signal strength arriving at a receiver based on the Friis transmission equation. It uses the transmit power 1 W, transmit antenna gain 10, receive antenna gain 5, operating frequency 2.4 GHz, and link distance 1 km. The resulting received power is expressed as 3.12463e-09 W in watts and -55.052 in decibels relative to one milliwatt. This value indicates whether the communication link has sufficient signal strength for reliable operation under free space conditions.

Radar (3)

Doppler shift of a radar echo — `radar_doppler`

The two-way Doppler shift of the echo from a target closing at radial velocity v (negative when receding): fd = 2 v f / (c - v), essentially 2 v / lambda.

$$f_d = \frac{2vf}{c - v} \approx \frac{2v}{\lambda}$$

symbol		meaning	SI unit
`v`	in	radial velocity, positive when approaching	m/s
`f`	in	transmitted frequency	Hz
`fd`	out	Doppler shift of the echo	Hz
`wavelength`	out	wavelength	m

Applies when: monostatic radar; |v| much smaller than c.

```
$ signal-processing solve radar_doppler "v=30 m/s" "f=10 GHz"
fd = 2001.384771 Hz
wavelength = 0.0299792458 m
  by: Doppler shift of a radar echo  (two-way Doppler of a reflector: f_r = f (c + v)/(c - v))
  v = 30 m/s
  f = 1e+10 Hz
  assumes: monostatic radar; |v| much smaller than c
```

What the worked example means: This calculation determines the two-way Doppler frequency shift of a radar echo based on the target's motion. Using a transmitted frequency of 10 GHz and a radial velocity of 30 m/s, the resulting Doppler shift is 2001.38 Hz. The corresponding wavelength of the signal is 0.0299792 m. This positive shift indicates that the target is approaching the radar, allowing for velocity measurement by comparing the returned frequency to the original.

Maximum detection range of a radar — `radar_max_range`

The largest range R = (Pt G^2 lambda^2 sigma / ((4 pi)^3 Smin))^(1/4) at which the echo still reaches the minimum detectable signal.

$$R_{\max} = \left(\frac{P_t G^2 \lambda^2 \sigma}{(4\pi)^3 S_{\min}} \right)^{1/4}$$

symbol		meaning	SI unit
`Pt`	in	peak transmit power	W
`G_db`	in	antenna gain in dBi	—
`f`	in	frequency	Hz
`sigma`	in	radar cross-section	m^2
`Pmin`	in	minimum detectable signal power	W
`Rmax`	out	maximum detection range	m

Applies when: monostatic radar; free space, no losses.

```
$ signal-processing solve radar_max_range "Pmin=1e-13 W" "Pt=1 MW" "sigma=1 m^2" G_db=40 "f=3 GHz"
Rmax = 266343.6371 m
  by: Maximum detection range of a radar  (radar range equation solved for R)
  Pmin = 1e-13 W
  Pt = 1000000 W
  sigma = 1 m^2
  G_db = 40
  f = 3000000000 Hz
  assumes: monostatic radar; free space, no losses
```

What the worked example means: This calculation determines the furthest distance at which a target can be detected, given the peak transmit power of 1 MW, antenna gain in dBi of 40, frequency of 3 GHz, radar cross-section of 1 m^2, and minimum detectable signal power of 1e-13 W. The resulting maximum range is 266344 m. This value represents the limit where the returned echo strength equals the system's noise floor or sensitivity threshold.

Received echo power (radar range equation) — `radar_received_power`

The echo power Pr = Pt G^2 lambda^2 sigma / ((4 pi)^3 R^4) of a monostatic radar from a target of cross-section sigma at range R.

$$P_r = \frac{P_t G^2 \lambda^2 \sigma}{(4\pi)^3 R^4}$$

symbol		meaning	SI unit
`Pt`	in	peak transmit power	W
`G_db`	in	antenna gain in dBi (transmit and receive)	—
`f`	in	frequency	Hz
`sigma`	in	radar cross-section	m^2
`R`	in	range to the target	m
`Pr`	out	received echo power	W
`Pr_dbm`	out	received echo power in dBm	—

Applies when: monostatic radar, one antenna; free space, no losses; far field.

```
$ signal-processing solve radar_received_power "Pt=1 MW" "sigma=1 m^2" G_db=40 "f=3 GHz" "R=100 km"
Pr = 5.032332221e-12 W
Pr_dbm = -82.98230696
  by: Received echo power (radar range equation)  (radar range equation)
  Pt = 1000000 W
  sigma = 1 m^2
  G_db = 40
  f = 3000000000 Hz
  R = 100000 m
  assumes: monostatic radar, one antenna; free space, no losses; far field
```

In the worked example, Pt = 1 MW, sigma = 1 m^2, G_db = 40, f = 3 GHz, R = 100 km go through the equation above and give Pr = 5.032332221e-12 W, Pr_dbm = -82.98230696. It applies only when monostatic radar, one antenna; free space, no losses; far field: outside that it says what is wrong instead of returning a number.

Sampling and signal basics (7)

Alias (apparent) frequency after sampling — `alias_frequency`

The frequency in [0, fs/2] at which a sinusoid of frequency f appears after sampling at rate fs.

$$f_a = |f - f_s \,\mathrm{round}(f/f_s)|$$

symbol		meaning	SI unit
`f`	in	frequency of the sampled sinusoid	Hz
`fs`	in	sampling rate	Hz
`f_alias`	out	apparent (principal alias) frequency	Hz

Applies when: f >= 0; fs > 0.

```
$ signal-processing solve alias_frequency "fs=10 kHz" "f=7 kHz"
f_alias = 3000 Hz
  by: Alias (apparent) frequency after sampling  (fold f about multiples of fs: |f -
fs*round(f/fs)|)
  fs = 10000 Hz
  f = 7000 Hz
  assumes: f >= 0; fs > 0
```

What the worked example means: This calculation determines the principal alias frequency, which is the value in the range between zero and half the sampling rate at which a sinusoid appears after being sampled. Taking the absolute difference between the true signal frequency 7 kHz and the nearest integer multiple of the sampling rate 10 kHz shows that the observed frequency is 3000 Hz. This result explains why high-frequency components can masquerade as lower-frequency signals in digital systems, potentially leading to misinterpretation of spectral data if not accounted for during analysis.

Frequency and angular frequency from period — `frequency_from_period`

The frequency f = 1/T and the angular frequency omega = 2*pi/T of a periodic signal of period T.

$$f = \frac{1}{T}, \quad \omega = \frac{2\pi}{T}$$

symbol		meaning	SI unit
`T`	in	period of the signal	s
`f`	out	frequency	Hz
`omega`	out	angular frequency	rad/s

Applies when: T > 0.

```
$ signal-processing solve frequency_from_period "T=2 ms"
f = 500 Hz
omega = 3141.592654 rad/s
  by: Frequency and angular frequency from period  (f = 1/T, omega = 2 pi / T)
  T = 0.002 s
  assumes: T > 0
```

What the worked example means: Given a period of 2 ms, the calculation determines the signal's repetition rate in hertz and its rotational speed in radians per second. The resulting frequency of 500 Hz indicates how many cycles occur each second, while the angular frequency of 3141.59 rad/s describes the phase change per unit time.

Mean, RMS, energy and average power of a sampled signal — `signal_statistics`

For samples x[0..N-1]: the mean, the RMS value, the energy sum |x|^2 and the average power (energy / N).

$$E = \sum_{n} |x[n]|^2, \quad P = \frac{E}{N}, \quad x_{\mathrm{rms}} = \sqrt{P}$$

symbol		meaning	SI unit
`x`	in	samples of the signal	—
`rms`	out	root-mean-square value	—
`mean`	out	mean (DC) value	—
`energy`	out	energy: sum of x[n]^2	—
`power`	out	average power: energy / N	—
`peak`	out	peak absolute value	—

Applies when: at least one sample.

```
$ signal-processing solve signal_statistics "x=[1, -2, 3, 0]"
mean = 0.5
rms = 1.870828693
energy = 14
power = 3.5
peak = 3
  by: Mean, RMS, energy and average power of a sampled signal  (direct sums (compensated))
  assumes: at least one sample
```

What the worked example means: The fundamental statistical properties were computed for the provided samples. The total energy is the sum of the squared magnitudes of all samples, resulting in 14. Dividing this energy by the number of samples yields the average power, which is 3.5. The root-mean-square value, representing the effective magnitude of the signal, is calculated as the square root of the average power, giving 1.87083. Additionally, the arithmetic mean of the samples, indicating the DC offset, is 0.5, while the peak absolute value observed is 3.

Nyquist rate — `nyquist_rate`

The minimum sampling rate 2*f_max that avoids aliasing of a baseband signal whose highest frequency is f_max.

$$f_{s,\min} = 2f_{\max}$$

symbol		meaning	SI unit
`fmax`	in	highest frequency in the signal	Hz
`fs_min`	out	Nyquist rate (minimum sampling rate)	Hz
`Ts_max`	out	longest sampling interval	s

Applies when: fmax > 0; a baseband (lowpass) signal.

```
$ signal-processing solve nyquist_rate "fmax=4 kHz"
fs_min = 8000 Hz
Ts_max = 0.000125 s
  by: Nyquist rate  (sampling theorem: fs >= 2 fmax)
  fmax = 4000 Hz
  assumes: fmax > 0; a baseband (lowpass) signal
```

What the worked example means: This calculation determines the Nyquist rate, defined as twice the maximum frequency present in a baseband signal. Given an input highest frequency of 4 kHz, the required minimum sampling rate is computed as 8000 Hz. This threshold ensures that the original analog signal can be perfectly reconstructed from its samples without spectral overlap or aliasing artifacts.

Period and angular frequency from frequency — `period_from_frequency`

The period T = 1/f and the angular frequency omega = 2*pi*f of a periodic signal of frequency f.

$$T = \frac{1}{f}, \quad \omega = 2\pi f$$

symbol		meaning	SI unit
`f`	in	frequency of the signal	Hz
`T`	out	period	s
`omega`	out	angular frequency	rad/s

Applies when: f > 0.

```
$ signal-processing solve period_from_frequency "f=50 Hz"
T = 0.02 s
omega = 314.1592654 rad/s
  by: Period and angular frequency from frequency  (T = 1/f, omega = 2 pi f)
  f = 50 Hz
  assumes: f > 0
```

What the worked example means: This calculation determines the time required for one complete cycle of a signal, known as the period, and the rate of phase change, known as the angular frequency. Starting from the input frequency of 50 Hz, which represents the number of cycles per second, it follows that the period is 0.02 s seconds. This means it takes 0.02 s to complete one repetition. Simultaneously, the angular frequency is calculated as 314.159 rad/s radians per second, indicating how fast the signal's phase rotates. These values translate the linear frequency into temporal and rotational domains, essential for analyzing oscillatory behavior.

RMS, peak-to-peak and rectified average of a sine wave — `sine_rms`

For a sinusoid of peak amplitude Vp: the RMS value Vp/sqrt(2), the peak-to-peak value 2*Vp and the average of its absolute value 2*Vp/pi.

$$V_{\mathrm{rms}} = \frac{V_p}{\sqrt{2}}$$

symbol		meaning	SI unit
`Vp`	in	peak amplitude of the sinusoid	V
`Vrms`	out	RMS value	V
`Vpp`	out	peak-to-peak value	V
`Vavg`	out	average of the rectified wave	V

Applies when: Vp > 0; a pure sinusoid with no DC offset.

```
$ signal-processing solve sine_rms "Vp=10 V"
Vrms = 7.071067812 V
Vpp = 20 V
Vavg = 6.366197724 V
  by: RMS, peak-to-peak and rectified average of a sine wave  (V_rms = V_p / sqrt(2))
  Vp = 10 V
  assumes: Vp > 0; a pure sinusoid with no DC offset
```

What the worked example means: This calculation determines three key magnitude metrics for a pure sinusoidal signal based on its peak amplitude 10 V. The root mean square value, representing the equivalent DC power delivery, is computed as 7.07107 V. The total vertical excursion from minimum to maximum, known as the peak-to-peak value, is found to be 20 V. Additionally, the average of the absolute voltage over one cycle, which reflects the rectified signal strength, results in 6.3662 V. These values allow engineers to characterize the energy content and visual range of the waveform.

Signal-to-quantization-noise ratio of an ideal ADC — `quantization_snr`

The SQNR of an ideal N-bit uniform quantizer driven by a full-scale sine wave (about 6.02 N + 1.76 dB).

$$\mathrm{SQNR} = 10\log_{10}\left(\tfrac{3}{2}\,2^{2N}\right) \approx 6.02N + 1.76\ \mathrm{dB}$$

symbol		meaning	SI unit
`bits`	in	resolution in bits	—
`sqnr_db`	out	signal-to-quantization-noise ratio in dB	—
`dynamic_range_db`	out	ideal dynamic range 20 log10(2^N) in dB	—

Applies when: ideal uniform quantizer; full-scale sinusoidal input; noise uniformly distributed.

```
$ signal-processing solve quantization_snr bits=12
sqnr_db = 74.00811155
dynamic_range_db = 72.24719896
  by: Signal-to-quantization-noise ratio of an ideal ADC  (ratio of the sine's power A^2/2 to the
quantization noise power Delta^2/12 with Delta = 2A/2^N)
  assumes: ideal uniform quantizer; full-scale sinusoidal input; noise uniformly distributed
```

What the worked example means: This calculation determines the theoretical performance of an ideal analog-to-digital converter by comparing the power of a full-scale sine wave to the noise introduced by quantization. Using the input resolution of 12 bits, the model computes that the signal-to-quantization-noise ratio reaches 74.0081 dB. This value reflects the quality of the digital representation relative to the original continuous signal, while the associated dynamic range is calculated as 72.2472 dB, indicating the total span between the smallest and largest representable signal levels.

Transmission lines (6)

Characteristic impedance of a coaxial line — `coax_impedance`

The characteristic impedance (eta0 / (2 pi sqrt(er))) ln(D/d) of a lossless coaxial line and its velocity factor 1/sqrt(er).

$$Z_0 = \frac{\eta_0}{2\pi\sqrt{\varepsilon_r}} \ln \frac{D}{d}$$

symbol		meaning	SI unit
`D`	in	inner diameter of the outer conductor	m
`d`	in	diameter of the inner conductor	m
`er`	in	relative permittivity of the dielectric (1 if not given) (optional)	—
`Z0`	out	characteristic impedance	ohm
`vf`	out	velocity factor	—

Applies when: d < D; er >= 1; lossless conductors and dielectric.

```
$ signal-processing solve coax_impedance "D=3.5 mm" "d=1 mm" er=2.25
Z0 = 50.07585194 ohm
vf = 0.6666666667
  by: Characteristic impedance of a coaxial line  (Z0 = sqrt(L'/C') with L' = mu ln(D/d)/(2 pi), C'
= 2 pi eps / ln(D/d))
  D = 0.0035 m
  d = 0.001 m
  er = 2.25
  assumes: d < D; er >= 1; lossless conductors and dielectric
```

What the worked example means: This calculation determines the characteristic impedance of a coaxial line using its physical dimensions and dielectric properties. Given an outer conductor inner diameter of 3.5 mm, an inner conductor diameter of 1 mm, and a relative permittivity of 2.25, the resulting characteristic impedance is 50.0759 ohm. The velocity factor, which indicates how fast signals travel compared to the speed of light in a vacuum, is calculated as 0.666667. These values are critical for matching transmission lines to prevent signal reflections.

Input impedance of a terminated lossless line — `input_impedance`

The impedance seen at the input of a lossless line of characteristic impedance Z0 and electrical length l (in wavelengths) terminated in ZL = RL + jXL.

$$Z_{in} = Z_0 \frac{Z_L + jZ_0 \tan \beta l}{Z_0 + jZ_L \tan \beta l}$$

symbol		meaning	SI unit
`RL`	in	load resistance	ohm
`Z0`	in	characteristic impedance	ohm
`length`	in	line length in wavelengths	—
`XL`	in	load reactance (0 if not given) (optional)	ohm
`Rin`	out	input resistance	ohm
`Xin`	out	input reactance	ohm

symbol		meaning	SI unit
`abs_zin`	out	|Zin|	ohm

Applies when: lossless line; RL > 0; length >= 0 in wavelengths.

```
$ signal-processing solve input_impedance "RL=100 ohm" "XL=0 ohm" length=0.125 "Z0=50 ohm"
Rin = 40 ohm
Xin = -30 ohm
abs_zin = 50 ohm
  by: Input impedance of a terminated lossless line  (Gamma moved along the line: Gamma_in =
Gamma_L e^{-j 4 pi l / lambda}, Zin = Z0 (1 + Gamma_in)/(1 - Gamma_in))
  RL = 100 ohm
  XL = 0 ohm
  length = 0.125
  Z0 = 50 ohm
  assumes: lossless line; RL > 0; length >= 0 in wavelengths
```

What the worked example means: This calculation determines the impedance seen at the start of a lossless transmission line. Using a characteristic impedance of 50 ohm and a length of 0.125 wavelengths, terminated by a load with resistance 100 ohm and reactance 0 ohm, the resulting input resistance is 40 ohm ohms and input reactance is -30 ohm ohms. The total magnitude of this transformed impedance is 50 ohm ohms.

Quarter-wave impedance transformer — `quarter_wave_transformer`

The characteristic impedance sqrt(RL Z0) of the quarter-wave section that matches a resistive load RL to a line Z0, and its physical length at frequency f.

$$Z_t = \sqrt{R_L Z_0}, \quad \ell = \frac{v_f c}{4f}$$

symbol		meaning	SI unit
`RL`	in	load resistance	ohm
`Z0`	in	impedance to match to	ohm
`f`	in	design frequency (optional)	Hz
`vf`	in	velocity factor of the section (default 1) (optional)	—
`Zt`	out	characteristic impedance of the transformer	ohm
`length`	out	physical length of the section (f given)	m

Applies when: resistive load; lossless lines; exact match at the design frequency only.

```
$ signal-processing solve quarter_wave_transformer "RL=100 ohm" "Z0=50 ohm" "f=1 GHz"
Zt = 70.71067812 ohm
length = 0.0749481145 m
  by: Quarter-wave impedance transformer  (Zin of a quarter-wave line is Zt^2 / RL; set it equal to
Z0)
  RL = 100 ohm
  Z0 = 50 ohm
  f = 1000000000 Hz
  assumes: resistive load; lossless lines; exact match at the design frequency only
```

What the worked example means: This calculation determines the characteristic impedance of a quarter-wave section needed to match a resistive load to a transmission line. The required impedance, 70.7107 ohm, is derived from the geometric mean of the load resistance and the system impedance. Additionally, the physical length of this section, 0.0749481 m, is computed based on the design frequency and the velocity factor of the medium.

Reflection coefficient and return loss from the VSWR — `reflection_from_vswr`

The reflection coefficient magnitude (VSWR - 1)/(VSWR + 1), the return loss and the mismatch loss for a given VSWR.

$$|\Gamma| = \frac{\mathrm{VSWR} - 1}{\mathrm{VSWR} + 1}$$

symbol		meaning	SI unit
`vswr`	in	voltage standing wave ratio (>= 1)	—
`gamma`	out	|Gamma|	—
`return_loss_db`	out	return loss in dB	—
`mismatch_loss_db`	out	mismatch loss in dB	—

Applies when: VSWR >= 1.

```
$ signal-processing solve reflection_from_vswr vswr=2
gamma = 0.3333333333
return_loss_db = 9.542425094
mismatch_loss_db = 0.5115252245
  by: Reflection coefficient and return loss from the VSWR  (|Gamma| = (VSWR - 1)/(VSWR + 1))
  vswr = 2
  assumes: VSWR >= 1
```

What the worked example means: This calculation determines the impedance match quality from the measured voltage standing wave ratio. Using the formula that relates these quantities shows that for an input VSWR of 2, the magnitude of the reflection coefficient is 0.333333. This corresponds to a return loss of 9.54243 and a mismatch loss of 0.511525. A lower return loss value indicates greater signal reflection at the interface.

Reflection coefficient, VSWR and return loss of a load — `reflection_coefficient`

For a load ZL = RL + jXL on a line of characteristic impedance Z0: the reflection coefficient (magnitude, angle, parts), VSWR, return loss and mismatch loss.

$$\Gamma = \frac{Z_L - Z_0}{Z_L + Z_0}, \quad \mathrm{VSWR} = \frac{1 + |\Gamma|}{1 - |\Gamma|}$$

symbol		meaning	SI unit
`RL`	in	load resistance	ohm
`Z0`	in	characteristic impedance of the line	ohm
`XL`	in	load reactance (0 if not given) (optional)	ohm

symbol		meaning	SI unit
`gamma_mag`	out	|Gamma|	—
`gamma_angle_deg`	out	angle of Gamma in degrees	—
`gamma_re`	out	real part of Gamma	—
`gamma_im`	out	imaginary part of Gamma	—
`vswr`	out	voltage standing wave ratio	—
`return_loss_db`	out	return loss in dB	—
`mismatch_loss_db`	out	mismatch loss in dB	—

Applies when: RL > 0; Z0 real (lossless line).

```
$ signal-processing solve reflection_coefficient "RL=75 ohm" "XL=25 ohm" "Z0=50 ohm"
gamma_mag = 0.2773500981
gamma_angle_deg = 33.69006753
gamma_re = 0.2307692308
gamma_im = 0.1538461538
vswr = 1.767591879
return_loss_db = 11.13943352
mismatch_loss_db = 0.3476210626
  by: Reflection coefficient, VSWR and return loss of a load  (Gamma = (ZL - Z0)/(ZL + Z0); VSWR =
(1 + |Gamma|)/(1 - |Gamma|))
  RL = 75 ohm
  XL = 25 ohm
  Z0 = 50 ohm
  assumes: RL > 0; Z0 real (lossless line)
```

What the worked example means: This calculation determines the impedance mismatch between a transmission line and its load. Given a load resistance of 75 ohm ohms and reactance of 25 ohm ohms connected to a line with a characteristic impedance of 50 ohm ohms, the reflection coefficient Gamma is computed as the ratio of the impedance difference to the sum. The resulting magnitude of Gamma is 0.27735, which directly translates to a Voltage Standing Wave Ratio (VSWR) of 1.76759. A lower VSWR indicates better power transfer efficiency, while higher values suggest significant signal reflection. The return loss, measured at 11.1394 dB, quantifies how much less reflected power there is compared to incident power.

VSWR and return loss from the reflection coefficient — `vswr_from_reflection`

The VSWR (1 + |Gamma|)/(1 - |Gamma|), the return loss -20 log10 |Gamma| and the mismatch loss from the magnitude of the reflection coefficient.

$$\mathrm{VSWR} = \frac{1 + |\Gamma|}{1 - |\Gamma|}$$

symbol		meaning	SI unit
`gamma`	in	magnitude of the reflection coefficient (0 <= |Gamma| < 1)	—
`vswr`	out	voltage standing wave ratio	—
`return_loss_db`	out	return loss in dB	—
`mismatch_loss_db`	out	mismatch loss in dB	—

Applies when: 0 <= |Gamma| < 1.

```
$ signal-processing solve vswr_from_reflection gamma=0.2
vswr = 1.5
return_loss_db = 13.97940009
mismatch_loss_db = 0.1772876696
  by: VSWR and return loss from the reflection coefficient  (VSWR = (1 + |Gamma|)/(1 - |Gamma|))
  gamma = 0.2
  assumes: 0 <= |Gamma| < 1
```

What the worked example means: This calculation determines the voltage standing wave ratio, return loss in decibels, and mismatch loss in decibels from the magnitude of the reflection coefficient. Given an input reflection coefficient magnitude of 0.2, the resulting voltage standing wave ratio is 1.5, indicating the degree of impedance mismatch. The return loss of 13.9794 quantifies the power reflected back toward the source, while the mismatch loss of 0.177288 represents the reduction in available power delivered to the load due to this reflection.

Wavelets (2)

Haar discrete wavelet transform — `haar_dwt`

The multi-level orthonormal Haar wavelet transform of a sequence whose length is a power of two: the final approximation and the detail coefficients of every level (coarsest first).

$$c_A[k] = \frac{x[2k] + x[2k+1]}{\sqrt{2}}, \quad c_D[k] = \frac{x[2k] - x[2k+1]}{\sqrt{2}}$$

symbol		meaning	SI unit
`x`	in	samples (length a power of two)	—
`levels`	in	number of levels (default 1) (optional)	—
`approximation`	out	approximation coefficients after the last level	—
`details`	out	detail coefficients per level, coarsest first	—

Applies when: length a power of two; levels <= log2 N.

```
$ signal-processing solve haar_dwt levels=2 "x=[4, 6, 10, 12, 8, 6, 5, 5]"
approximation = [16.000000000000004,12.000000000000002]
details = [[-6.000000000000001,2.0],
[-1.4142135623730951,-1.4142135623730951,1.4142135623730951,0.0]]
  by: Haar discrete wavelet transform  (repeated pairwise (a + b)/sqrt 2 and (a - b)/sqrt 2)
  assumes: length a power of two; levels <= log2 N
```

What the worked example means: This calculation applies the multi-level orthonormal Haar discrete wavelet transform to the input sequence [4, 6, 10, 12, 8, 6, 5, 5], using 2 decomposition levels. The method repeatedly pairs adjacent samples to compute averages for the approximation coefficients and differences for the detail coefficients, scaled by one over the square root of two. The result provides a coarse trend representation in [16, 12] and scale-specific fluctuation data in [[-6, 2], [-1.41421, -1.41421, 1.41421, 0]], where details are ordered from the coarsest level first.

Inverse Haar wavelet transform (one level) — `haar_idwt`

The sequence reconstructed from one level of Haar approximation and detail coefficients.

$$x[2k] = \frac{c_A[k] + c_D[k]}{\sqrt{2}}, \quad x[2k+1] = \frac{c_A[k] - c_D[k]}{\sqrt{2}}$$

symbol		meaning	SI unit
`cA`	in	approximation coefficients	—
`cD`	in	detail coefficients	—
`x`	out	reconstructed samples	—

Applies when: equal lengths.

```
$ signal-processing solve haar_idwt "cA=[16, 12]" "cD=[-6, 2]"
x = [7.0710678118654755,15.556349186104047,9.899494936611665,7.0710678118654755]
  by: Inverse Haar wavelet transform (one level)  (x[2k] = (a + d)/sqrt 2, x[2k+1] = (a - d)/sqrt
2)
  assumes: equal lengths
```

What the worked example means: This calculation performs the inverse Haar wavelet transform on a single level to reconstruct the original time-domain samples. It combines the approximation coefficients [16, 12] with the detail coefficients [-6, 2] using specific linear operations to produce the reconstructed signal [7.07107, 15.5563, 9.89949, 7.07107]. The process effectively reverses the decomposition step, merging low-frequency trend information and high-frequency fluctuation data back into their original form.

Windows and spectral analysis (3)

Coefficients of a window function — `window_coefficients`

The samples w[0..N-1] of a rectangular, Hann, Hamming or Blackman window (symmetric for filter design, periodic for spectral analysis) and its coherent gain.

$$w[n] = \sum_k (-1)^k a_k \cos \frac{2\pi k n}{M}$$

symbol		meaning	SI unit
`window`	in	window type	—
`N`	in	window length	—
`form`	in	symmetric (default) or periodic (optional)	—
`w`	out	window samples w[0..N-1]	—
`coherent_gain`	out	coherent gain: mean of w	—

Applies when: N >= 2.

```
$ signal-processing solve window_coefficients window=hann form=periodic N=4
w = [0.0,0.49999999999999994,1.0,0.5000000000000001]
coherent_gain = 0.5
  by: Coefficients of a window function  (generalized cosine sums: w[n] = sum_k (-1)^k a_k cos(2 pi
k n / M), M = N-1 (symmetric) or N (periodic))
  form = periodic
  window = hann
  assumes: N >= 2
```

What the worked example means: This calculation produces the time-domain coefficients for the specified window type, such as Hann or Blackman, based on the length 4 and form periodic. The resulting samples, denoted as [0, 0.5, 1, 0.5], represent the amplitude weighting applied to signal segments. The coherent gain, shown as 0.5, indicates the average amplitude scaling factor introduced by this windowing process.

Main-lobe width of a spectral window — `window_mainlobe_width`

The null-to-null main-lobe width of the rectangular, Hann, Hamming or Blackman window (DFT-even form), in bins and, with fs, in hertz.

$$\Delta f_{\mathrm{ML}} = k\,\frac{f_s}{N},\ k = 2\ (\mathrm{rect}),\ 4\ (\mathrm{Hann, Hamming}),\ 6\ (\mathrm{Blackman})$$

symbol		meaning	SI unit
`window`	in	window type	—
`N`	in	window length	—
`fs`	in	sampling rate (for the width in Hz) (optional)	Hz
`width_bins`	out	null-to-null main-lobe width in DFT bins	—
`first_null_bins`	out	position of the first null in bins	—
`width_hz`	out	main-lobe width in Hz (fs given)	Hz

Applies when: N >= 8; DFT-even (periodic) windows for spectral analysis.

```
$ signal-processing solve window_mainlobe_width window=hann N=256 "fs=8 kHz"
width_bins = 4
first_null_bins = 2
width_hz = 125 Hz
  by: Main-lobe width of a spectral window  (tabulated widths 2, 4, 4, 6 bins (the first zero of
each window's transform), times fs/N)
  fs = 8000 Hz
  window = hann
  assumes: N >= 8; DFT-even (periodic) windows for spectral analysis
```

What the worked example means: This calculation determines the frequency resolution limit imposed by the chosen spectral window. The main-lobe width, defined as the distance between the first two zeros of the window's transform, is 4 bins wide. When expressed in physical frequency units using the sampling rate, this width corresponds to 125 Hz hertz. A wider main lobe indicates lower ability to distinguish closely spaced sinusoidal components, while a narrower lobe suggests better resolution but potentially higher side-lobe levels depending on the specific window type selected.

Periodogram (power spectral density estimate) — `periodogram`

The one-sided periodogram PSD estimate (rectangular window, mean removed) of a sequence sampled at fs, its frequency grid and the frequency of its peak.

$$P[k] = \frac{2\,|X[k]|^2}{f_s N}\ (0 < k < N/2)$$

symbol		meaning	SI unit
`x`	in	samples	—
`fs`	in	sampling rate	Hz
`peak_frequency`	out	frequency of the largest PSD value	Hz
`freqs`	out	frequencies k fs / N, k = 0 .. N/2	—
`psd`	out	one-sided power spectral density (power per hertz)	—

Applies when: mean removed first; rectangular window; one-sided density scaling.

```
$ signal-processing solve periodogram "x=[1, 0, -1, 0, 1, 0, -1, 0]" "fs=8 Hz"
peak_frequency = 2 Hz
freqs = [0.0,1.0,2.0,3.0,4.0]
psd = [0.0,0.0,0.5,0.0,0.0]
  by: Periodogram (power spectral density estimate)  (FFT of the mean-removed samples; |X[k]|^2 /
(fs N), doubled except at DC and Nyquist)
  fs = 8 Hz
  assumes: mean removed first; rectangular window; one-sided density scaling
```

What the worked example means: This calculation estimates the one-sided power spectral density by applying a fast Fourier transform to the mean-removed input samples and scaling the squared magnitude. The resulting frequency grid spans up to half the sampling rate, with the highest energy concentration found at 2 Hz Hz. This provides a non-parametric view of how signal power is distributed across frequencies for the given sequence.

Z-transform and stability (2)

Poles and stability of a second-order all-pole system — `second_order_poles`

The poles of H(z) = 1/(1 + a1 z^-1 + a2 z^-2) (roots of z^2 + a1 z + a2), their largest radius and whether the causal system is stable.

$$z^2 + a_1 z + a_2 = 0, \quad \text{stable} \iff |p_i| < 1$$

symbol		meaning	SI unit
`a1`	in	coefficient of z^-1 in the denominator	—
`a2`	in	coefficient of z^-2 in the denominator	—
`radius`	out	largest pole radius |p|	—
`p1`	out	first pole (smaller real, or positive imaginary part)	—
`p2`	out	second pole	—

symbol		meaning	SI unit
`stability`	out	stable if every pole lies inside the unit circle	—

Applies when: a2 != 0; a causal system (stable iff |p| < 1 for every pole).

```
$ signal-processing solve second_order_poles a1=-1.2 a2=0.5
radius = 0.7071067812
p1 = {"re":0.6,"im":0.37416573867739417}
p2 = {"re":0.6,"im":-0.37416573867739417}
stability = stable
  by: Poles and stability of a second-order all-pole system  (roots of z^2 + a1 z + a2 by the
numerically stable quadratic formula)
  a1 = -1.2
  a2 = 0.5
  assumes: a2 != 0; a causal system (stable iff |p| < 1 for every pole)
```

What the worked example means: This calculation determines the poles of a second-order all-pole filter by solving the characteristic equation. The resulting pole locations are 0.6 + 0.374166i and 0.6 - 0.374166i, with a maximum magnitude of 0.707107. Since stability requires every pole to lie strictly inside the unit circle, the system is classified as stable based on these values.

Poles, zeros and stability of a digital filter — `poles_zeros`

The zeros and poles of H(z) = sum b_k z^-k / sum a_k z^-k (up to 4th order), the largest pole radius and whether the causal filter is stable.

$$H(z) = \frac{\sum_k b_k z^{-k}}{\sum_k a_k z^{-k}} = \frac{b_0 \prod (1 - z_i z^{-1})}{a_0 \prod (1 - p_i z^{-1})}$$

symbol		meaning	SI unit
`b`	in	numerator coefficients b (up to 5)	—
`a`	in	denominator coefficients a (up to 5, a[0] != 0)	—
`max_pole_radius`	out	largest pole radius	—
`zeros`	out	zeros (real ascending, then complex)	—
`poles`	out	poles (real ascending, then complex)	—
`stability`	out	stable if every pole lies inside the unit circle	—

Applies when: a[0] != 0; orders up to 4; both polynomials in powers of z^-1 (padded to equal length).

```
$ signal-processing solve poles_zeros "b=[1, -1]" "a=[1, -0.9]"
max_pole_radius = 0.9
zeros = [1.0]
poles = [0.9]
stability = stable
  by: Poles, zeros and stability of a digital filter  (polynomial roots (closed form up to degree
2; Durand-Kerner iteration with Newton polishing above))
  assumes: a[0] != 0; orders up to 4; both polynomials in powers of z^-1 (padded to equal length)
```

What the worked example means: This analysis computes the zeros and poles from the numerator coefficients [1, -1] and denominator coefficients [1, -0.9]. The resulting poles are listed in [0.9] and zeros in [1]. Stability is determined by checking if every pole lies within the unit circle, which depends on the largest pole radius being less than one. The calculated maximum pole radius is 0.9, leading to the conclusion that the system is stable.

Questions and reports: support@ai999x.com. AI999X — The Future Is Self-Improving.